VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi-590018



A PROJECT REPORT

On

"ForgeXplain: An Explainable Machine Learning System for Offline Signature Forgery Detection"

A Dissertation Submitted in partial fulfillment of the requirement for the degree of

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by

MADIHA KAUNAIN (1RG23CS026)

NALINI M (1RG23CS040)

NOOR AYESHA (1RG23CS045)

Under The Guidance of

Mrs. R. Saranya M. E

Assistant Professor, Dept. of CSE

RGIT, Bengaluru-32



Department of Computer Science & Engineering

RAJIV GANDHI INSTITUTE OF TECHNOLOGY

Cholanagar, R. T. Nagar Post, Bengaluru-560032

2026-27

RAJIV GANDHI INSTITUTE OF TECHNOLOGY

(Affiliated to Visvesvaraya Technological University)

Cholanagar, R. T. Nagar Post, Bengaluru-560032

Department of Computer Science & Engineering

CERTIFICATE

Phase – II

This is to certify that the Project Report entitled **"ForgeXplain: An Explainable Machine Learning System for Offline Signature Forgery Detection"** is a Bonafide work carried out by **Ms. MADIHA KAUNAIN (1RG22CS026), Ms. NALINI M (1RG22CS040), Ms. NOOR AYESHA (1RG22CS045)**, in partial fulfillment for the award of **Bachelor of Engineering** in **Computer Science and Engineering** under **Visvesvaraya Technological University, Belagavi,** during the year **2026-2027**. It is certified that all corrections/suggestions given for Internal Assessment have been incorporated in the report. This Project report has been approved as it satisfies the academic requirements in respect of project work (BCS786) prescribed for the said degree.

Signature of guide	Signature of HOD	Signature of Principal
Mrs. R. Sharanya M.E	**Dr. Arudra A**	**Dr. D G Anand**
Assistant Professor	Associate Professor, HOD	Principal, RGIT
Dept. of CSE	Dept. of CSE	Bengaluru-32
RGIT, Bengaluru-32	RGIT, Bengaluru-32	

External Viva

Name of Examiners	Signature with date
1.	
2.	

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi-590018

RAJIV GANDHI INSTITUTE OF TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

DECLARATION

We hereby declare that the project work entitled **"ForgeXplain: An Explainable Machine Learning System for Offline Signature Forgery Detection"** submitted to the **Visvesvaraya Technological University, Belagavi** during the academic year **2026-2027**, is record of an original work done by us under the guidance of M**rs. R. Saranya M. E** , Assistant professor, Rajiv Gandhi Institute of Technology , Bengaluru and this project work is submitted in the partial fulfillment of requirements for the award of the degree of **Bachelor of Engineering in Computer Science & Engineering.** The results embodied in this project have not been submitted to any other University or Institute for award of any degree or diploma.

MADIHA KAUNAIN (1RG23CS026)

NALINI M (1RG23CS040)

NOOR AYESHA (1RG23CS045)

ACKNOWLEDGEMENT

We take this opportunity to express our sincere gratitude and respect to the **Rajiv Gandhi Institute of Technology, Bengaluru** for providing us an opportunity to carry out our project work. We express our sincere regards and thanks to **Dr. D G ANAND**, Principal, RGIT, Bengaluru. Also we would like to thanks **Dr. ARUDRA A**, Associate Professor and HOD, Department of Computer Science & Engineering, RGIT, Bengaluru, for their encouragement and support throughout the Project.

With profound sense of gratitude, we acknowledge the guidance and support extended towards us by project coordinators **Mrs. SONIYA KOMAL V**, Assistant Professor, Department of CSE, RGIT, Bengaluru. **Mrs. BHAGYASHRI WAKDE**, Assistant Professor, Department of CSE, RGIT, Bengaluru. Their incessant encouragement and valuable technical support have been of immense help in realizing this project. Their guidance gave us the environment to enhance our knowledge, skills and to reach the pinnacle with sheer determination, dedication and hard work.

We extend our heartfelt gratitude to **Mrs. R. SARANYA M. E**, Assistant professor, Department of Computer Science & Engineering, RGIT, Bengaluru, for their unwavering support and guidance in our project. Their expertise has been instrumental in shaping its success. We also appreciate the collaborative efforts of other coordinators and team members for enriching this learning experience.

We also extend our thanks to the entire faculty of the Department of CSE, RGIT, Bengaluru, who have encouraged throughout the course of Bachelor Degree. We extend our heartfelt thanks to our family for their full support and unwavering encouragement, which provided us with a safe and conducive environment for the completion of this project. Their assistance and belief in our abilities have been instrumental in this achievement.

MADIHA KAUNAIN (1RG23CS026)

NALINI M (1RG23CS040)

NOOR AYESHA (1RG23CS045)

ABSTRACT

Offline signature verification is an important task in forensic analysis and document authentication, where distinguishing genuine signatures from forged signatures can be challenging due to variations in writing style, image quality, and forgery techniques. Traditional signature verification methods may provide accurate results but often lack transparency in explaining how a particular decision is made.To address this limitation, ForgeXplain proposes an explainable machine learning framework for offline signature forgery detection. The system accepts a scanned signature image and performs preprocessing operations such as grayscale conversion, noise reduction, adaptive thresholding, morphological operations, and signature region extraction. A set of 23 interpretable features based on geometric, shape, density, contour, and intensity characteristics is extracted from the processed image.These features are classified using Support Vector Machine (SVM) and Random Forest (RF) algorithms. SHAP (SHapley Additive exPlanations) is incorporated to explain the model predictions by identifying the contribution of individual features to the final decision. The system is implemented as a Streamlit-based web application that provides signature analysis and visual explanations through an interactive interface.Experimental evaluation using the CEDAR offline signature dataset shows that the Random Forest model achieves an accuracy of 91.5% and a ROC-AUC of 0.966, outperforming the SVM model. The proposed ForgeXplain framework combines effective forgery detection with interpretable machine learning, making the prediction process easier to understand and supporting reliable applications in forensic analysis and document authentication.

CONTENTS

ACKNOWLEDGEMENT		**i**
ABSTRACT		**ii**
LIST OF FIGURES		**vii**
LIST OF TABLES		**viii**
CHAPTER NO	**TITLE**	**PAGE NO**
1	**INTRODUCTION**	**1**
1.1	**Overview**	**1**
1.2	**Motivation**	**4**
1.3	**Problem Identification**	**4**
1.4	**Scope of the Project**	**5**
1.5	**Objective and Methodology**	**6**
1.6	**Existing System**	**6**
1.7	**Proposed System**	**7**
1.8	**Outcome of the Project**	**9**
1.9	**Report Organization**	**9**
1.10	**Introduction Summary**	**10**
2	**LITERATURE SURVEY**	**11**
2.1	**Related Work**	
	2.1.1 Reference Papers	**11**
	2.1.2 Traditional Feature-Based Signature Verification	**11**
	2.1.3 Machine Learning-Based Offline Signature Verification	**12**
2.2	**Machine Learning-Based Offline Signature Verification Systems**	**12**

2.3	Deep Learning for Offline Handwritten Signature Verification	13
2.4	Offline Signature Verification Using Region-Based Deep Metric Learning	13
2.5	CEDAR Dataset and Signature-Based Classification	14
2.6	Hybrid Machine Learning Framework for Offline Signature Verification	15
2.7	Explainable AI in Signature Forgery Detection	15
2.8	Comparison of Existing Approaches with Forgexplain	16
2.9	Literature Survey Summary	17
3	SYSTEM ANALYSIS	18
3.1	Introduction to System Analysis	18
3.2	Feasibility Study	
	3.2.1 Technical Feasibility	19
	3.2.2 Economic Feasibility	19
	3.2.3 Operational Feasibility	20
	3.2.4 Schedule Feasibility	21
	3.2.5 Social Feasibility	22
3.3	Functional Requirements	23
3.4	Non-Functional Requirements	25
3.5	System Analysis Summary	27
4	REQUIREMENT ANALYSIS	28
4.1	Functional Requirements	28
4.2	Non-Functional Requirements	30
4.3	Software Requirements	

	4.3.1 Operating System	34
4.4	Software Requirement Summary	35
4.5	Hardware Requirements	36
4.6	Hardware Requirements Summary	37
4.7	Requirement Analysis Summary	38
5	SYSTEM DESIGN	39
5.1	System Architecture Overview	
	5.1.1 System Flow Design	41
	5.1.2 Design Characteristics	43
5.2	Gantt Chart	
	5.2.1 Gantt Chart of Phase One	44
	5.2.2 Gantt Chart of Phase Two	45
5.3	Life Cycle Model	46
5.4	Data Flow Diagram	48
5.5	Use Case Diagram	50
5.6	Sequence Diagram	51
5.7	Class Diagram	53
6	SYSTEM IMPLEMENTATION	56
6.1	Modular Approach	
	6.1.1 User Interface Module	56
	6.1.2 Image Processing Module	57
	6.1.3 Feature Extraction Module	57
	6.1.4 Machine Learning Classification Module	57
	6.1.5 SHAP Explanation Module	58
	6.1.6 Result Display Module	58
6.2	Programming Code	59
6.3	System Implementation Summary	65

7	**TESTING**	**66**
7.1	**Validation and System Testing**	
	7.1.1 Software Testing	**66**
	7.1.2 Validations	**68**
	7.1.3 Reasons for Performing Software Validation	**69**
7.2	**Testing Summary**	**70**
8	**SAMPLE OUTPUT**	**71**
8.1	**Snapshots**	**71**
8.2	**Sample Output Summary**	**74**

CONCLUSION

REFERENCES

LIST OF FIGURES/ILLUSTRATIONS

FIGURE NO.	FIGURE NAME	PAGE NO.
1.1	System Architecture	8
1.2	Workflow Diagram	9
5.1	System Architecture of ForgeXplain	41
5.2	System Flow Design of ForgeXplain	42
5.3	Gantt Chart of Phase One	45
5.4	Gantt Chart of Phase Two	46
5.5	Life Cycle Model of ForgeXplain	48
5.6	Data Flow Diagram of ForgeXplain	49
5.7	Use Case Diagram of ForgeXplain	51
5.8	Sequence Diagram of ForgeXplain	52
5.9	Class Diagram of ForgeXplain	55
8.1.1	ForgeXplain Login Page	71
8.1.2	ForgeXplain Home Dashboard	71
8.1.3	Signature Detection and Batch Processing Interface	72
8.1.4	Genuine Signature Detection Result	72
8.1.5	SHAP-Based Visual Explainability of Signature Prediction	73
8.1.6	Generated Forgexplain Forensic Audit Report	73

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
2.3	Deep Learning for Offline Handwritten Signature Verification	13
2.4	Offline Signature Verification Using Region-Based Deep Metric Learning	14
2.5	CEDAR Dataset and Signature-Based Classification	14
2.6	Hybrid Machine learning Framework for Offline Signature Verification	15
2.7	Explainable AI in Signature Forgery Detection	15
2.8	Comparison of Existing Approaches with ForgeXplain	16
4.5	Hardware Requirements	36
5.1	Phase One Project Activities	44
5.2	Phase Two Project Activities	44
5.3	Data Flow Description	49
7.1	Testing Results	67

CHAPTER 1

INTRODUCTION

1.1 Overview

Signatures are one of the most commonly used methods for verifying the identity and authenticity of a person in both physical and digital transactions. They are widely used in banking, financial institutions, legal documents, educational records, business agreements, cheque processing, and other official applications. A handwritten signature contains individual characteristics such as its shape, size, orientation, stroke pattern, spacing, and writing style. These characteristics can be used to distinguish a genuine signature from a forged one.

Traditional signature verification is generally performed manually by comparing a questioned signature with a known genuine signature. Although human experts can identify many types of forgery, manual verification is time-consuming, subjective, and difficult to apply when a large number of signatures have to be examined. The increasing use of scanned documents and digital records has created a need for automated methods that can analyze signature images efficiently.

Offline signature verification refers to the process of verifying a signature using a static image of a handwritten signature. Unlike online signature verification, it does not have access to dynamic information such as pen pressure, writing speed, stroke order, or pen movement. Therefore, the system must depend mainly on visual and structural characteristics present in the signature image.

Machine learning provides an effective approach for analyzing these characteristics. By extracting meaningful features from signature images and training classification models, a system can learn patterns associated with genuine and forged signatures. However, a prediction from a machine learning model alone may not explain why a particular signature has been classified as genuine or forged.

To address this limitation, ForgeXplain: Explainable Machine Learning Framework for Offline Signature Forgery Detection combines image preprocessing, interpretable feature extraction, machine learning classification, and Explainable Artificial Intelligence (XAI). The system accepts a scanned signature image, processes it to remove unwanted variations, extracts meaningful signature characteristics, and uses machine learning models such as Support Vector Machine (SVM) and Random Forest for classification. SHAP-based explanations are then used to show the contribution of individual features to the model's prediction.

The proposed system is implemented as a web-based application using Streamlit, providing an accessible interface for uploading signature images, obtaining predictions, and viewing feature-level explanations. This makes the system not only focused on classification accuracy but also on understanding and interpreting the decisions produced by the machine learning model.

1.1.1 Signature Verification in Real-World Applications

Signature verification helps confirm identity and approval, while automated systems reduce the effort required to check large numbers of signatures.

• Banking and Financial Transactions

Signatures are traditionally used in cheques, account-related documents, loan documents, and other financial records. Automated signature verification can help identify suspicious signatures and support the verification process.

• Legal and Official Documents

Contracts, agreements, applications, and other official documents may contain handwritten signatures. Signature verification can assist in checking whether a questioned signature resembles the authorized signature of an individual.

• Cheque and Document Processing

Financial institutions and organizations may need to process a large number of scanned documents. An automated system can analyze signature images and provide a preliminary classification, reducing the amount of manual checking required.

• Educational and Institutional Records

Signatures can be found on certificates, applications, attendance-related documents, permission forms, and other institutional records. Automated verification can provide an additional method for checking the authenticity of these signatures.

• Digital Document Verification

As organizations increasingly convert physical documents into digital formats, scanned signatures become an important source of information. Image-based signature verification can therefore be integrated into document-processing systems.

1.1.2 Challenges in Offline Signature Verification

Offline signature verification presents several challenges because the system receives only an image of the completed signature.

• Variations in Genuine Signatures

A person may not reproduce their signature exactly every time. Natural variations in size, position, stroke shape, and writing style can occur even when the signature is genuine.

• Different Types of Forgery

Forged signatures may be produced by copying the overall appearance of a genuine signature or by creating a signature that looks similar without accurately reproducing every characteristic. This makes the distinction between genuine and forged signatures difficult

• **Absence of Dynamic Information**

Offline signatures do not provide information such as writing speed, pen pressure, stroke order, or movement direction during writing. The system therefore has to rely only on information available in the static image.

• **Feature Selection**

A signature contains many visual characteristics. Selecting features that are meaningful for classification while remaining interpretable is an important challenge.

• **Interpretation of Predictions**

A machine learning model may classify a signature as genuine or forged, but the prediction alone does not explain which characteristics influenced the decision. This creates a need for explainable machine learning.

1.1.3 Machine Learning in Signature Forgery Detection

Machine learning techniques can be used to automatically identify patterns that distinguish genuine signatures from forged signatures. Instead of depending entirely on manually defined rules, machine learning models learn from previously available signature samples.

In ForgeXplain, the signature image is first subjected to preprocessing operations. These operations include resizing, conversion to grayscale, Gaussian filtering, adaptive thresholding, morphological operations, bounding-box extraction, and re-centering. The purpose of preprocessing is to obtain a cleaner and more consistent representation of the signature before feature extraction.

After preprocessing, meaningful characteristics are extracted from the signature image. The proposed framework uses **23 interpretable features** covering different aspects of the signature, including geometric, shape, density, contour, and intensity-histogram characteristics.

These extracted features are provided to machine learning classifiers. The proposed system uses **Support Vector Machine (SVM)** and **Random Forest (RF)** models to classify the signature. Their performance can be evaluated using measures such as accuracy, precision, recall, F1-score, and ROC-AUC.

The use of more than one classification model also allows the performance of different machine learning approaches to be examined within the same framework.

1.1.4 Explainable AI for Signature Verification

Although machine learning can provide automated predictions, understanding the reason behind a prediction is important in applications involving document and identity verification. A user may need to know which characteristics of a signature contributed to its classification.

Explainable Artificial Intelligence provides methods for understanding the decisions made by machine learning models. In ForgeXplain, SHAP (SHapley Additive exPlanations) is used to explain the contribution of individual features to a prediction.

The explanation identifies features that have influenced the classification and indicates how they contribute to the model's decision. For example, characteristics related to intensity distribution, shape, density, or geometric properties may have different levels of influence on the prediction.

This feature-level explanation makes the system more transparent than a classification system that provides only a final label. It can help users understand the model's reasoning and provides an additional layer of interpretability for offline signature forgery detection.

1.2 Motivation

Signatures continue to be used for authentication and approval in many important documents and transactions. However, manual signature verification becomes difficult when a large number of signatures have to be examined. The process can require considerable time and may also depend on the experience and judgment of the person performing the verification.

The availability of scanned documents creates an opportunity to automate this process using image processing and machine learning. However, an automated system should not only provide a prediction but should also provide information that helps users understand the basis of that prediction.

The motivation behind ForgeXplain is therefore to develop an integrated framework that combines offline signature image processing, interpretable feature extraction, machine learning classification, and explainable AI. The system is intended to make signature forgery detection more systematic while providing users with meaningful information about the features influencing the model's decision.

The use of a web-based Streamlit interface further simplifies interaction with the system. Users can provide a signature image and obtain the classification result along with its explanation through a single application.

1.3 Problem Identification

Despite advances in image processing and machine learning, several challenges remain in automated offline signature forgery detection.

Traditional manual verification requires human examination and may become inefficient when the number of signatures increases. Variations in genuine signatures can also make classification difficult because the same person may produce signatures with small differences.

Image quality is another challenge because scanned signature images may contain noise, background information, or differences in size and alignment. Without suitable preprocessing, these variations can affect the extracted features and consequently the classification result.

Another important limitation of conventional machine learning systems is the lack of interpretability. A classifier may provide a genuine or forged prediction without clearly showing which signature characteristics influenced that decision.

Therefore, there is a need for a system that can:

• Process and standardize offline signature images.

• Extract meaningful and interpretable signature features.

• Classify signatures as genuine or forged using machine learning.

• Compare the performance of different classification models.

• Explain individual predictions using an Explainable AI technique.

• Provide the complete process through an easy-to-use web interface.

ForgeXplain is designed to address these requirements by integrating all these stages into a unified framework.

1.4 Scope of the Project

The scope of this project is to design and develop an explainable machine learning framework for detecting forgery in offline signature images.

The system focuses on:

• Accepting scanned or uploaded signature images as input.

• Applying image preprocessing techniques to improve the quality and consistency of signature images.

• Extracting 23 interpretable features from the processed signature.

• Using geometric, shape, density, contour, and intensity-histogram characteristics for feature representation.

• Applying machine learning classifiers such as SVM and Random Forest.

• Classifying the input signature based on the learned patterns.

• Evaluating classification performance using standard machine learning metrics.

• Applying SHAP to explain the contribution of individual features.

• Providing prediction and explanation through a Streamlit-based web application.

The project focuses specifically on offline signature verification, where only the final signature image is available. Dynamic information such as pen pressure, stroke sequence, and writing speed is outside the scope of the system.

1.5 Objective and Methodology

The main objective of ForgeXplain is to develop an explainable machine learning framework capable of detecting forged offline signatures while providing an understandable explanation of the classification result.

The major objectives of the project are:

• To develop a preprocessing pipeline for offline signature images.

• To extract meaningful and interpretable features from signature images.

• To classify signatures using machine learning algorithms.

• To compare SVM and Random Forest classification approaches.

• To evaluate the performance of the models using standard evaluation metrics.

• To provide feature-level explanations using SHAP.

• To develop a user-friendly Streamlit web interface for signature analysis.

The methodology follows a sequential workflow. The user first uploads a signature image through the web interface. The image undergoes preprocessing operations such as resizing, grayscale conversion, Gaussian filtering, adaptive thresholding, morphological opening and closing, bounding-box extraction, and re-centering.

The processed signature is then represented using 23 interpretable features. These features are provided to the trained machine learning models, including SVM and Random Forest, which generate the classification result.

The model performance is evaluated using accuracy, precision, recall, F1-score, and ROC-AUC. Finally, SHAP is used to explain the contribution of individual features to the prediction. The complete workflow is integrated into the Streamlit application so that prediction and explanation can be obtained through a single interface.

1.6 Existing System

Existing signature verification methods generally depend on manual examination, traditional image-processing techniques, or machine learning systems that provide only a final classification result.

Manual verification requires a person to compare the questioned signature with a genuine reference signature. Although this approach can be useful, it becomes time-consuming when many signatures need to be examined.

Traditional automated approaches may use image-processing techniques to compare signatures based on their visual similarity. However, their performance can be affected by variations in image quality, signature size, alignment, and natural variations in genuine signatures.

Machine learning-based systems can automate the classification process, but some systems treat the classifier as a black box. They provide a prediction without clearly explaining which features contributed to the result.

Disadvantages

• **Time-consuming Manual Verification:** Manual comparison requires human effort and becomes difficult when a large number of signatures need to be checked.

• **Variation in Genuine Signatures:** A person's genuine signature can vary slightly from one occasion to another.

• **Image Quality Issues:** Noise, background variations, scanning conditions, and alignment differences can affect signature analysis.

• **Limited Interpretability:** Some machine learning systems provide only a classification result without explaining the reason behind the prediction.

• **Dependence on Manual Feature Analysis:** Conventional approaches may require significant effort to identify useful signature characteristics.

• **Fragmentation of Processing and Classification:** Image preprocessing, feature extraction, classification, and explanation may be handled as separate processes rather than through one integrated system.

1.7 Proposed System

The proposed system, **ForgeXplain**, introduces an explainable machine learning framework specifically designed for offline signature forgery detection. The system combines image preprocessing, interpretable feature extraction, machine learning classification, model evaluation, and Explainable AI into a single workflow.

The system provides a web-based interface developed using Streamlit through which users can upload a signature image for analysis. Once the image is provided, the system performs a sequence of preprocessing operations to obtain a standardized representation of the signature.

The preprocessing stage includes resizing, grayscale conversion, Gaussian filtering, adaptive Gaussian thresholding, morphological opening and closing, bounding-box extraction, and re-centering. These operations help reduce unwanted variations and prepare the signature image for feature extraction.

After preprocessing, the system extracts 23 interpretable features from the signature image. These features represent different properties of the signature, including geometric characteristics, shape information, density, contour properties, and intensity-histogram characteristics.

The extracted features are then provided to machine learning models. ForgeXplain uses Support Vector Machine (SVM) and Random Forest (RF) classifiers to identify whether the input signature belongs to the genuine or forged class. Model performance can be examined using accuracy, precision, recall, F1-score, and ROC-AUC.

A major feature of the proposed system is its use of SHAP-based Explainable AI. After generating a prediction, the system analyzes the contribution of individual features to the model's decision. This allows users to understand which characteristics of the signature influenced the classification rather than receiving only a final prediction.

The complete workflow is integrated into a Streamlit application, allowing image upload, preprocessing, prediction, and explanation to be performed through an accessible interface.

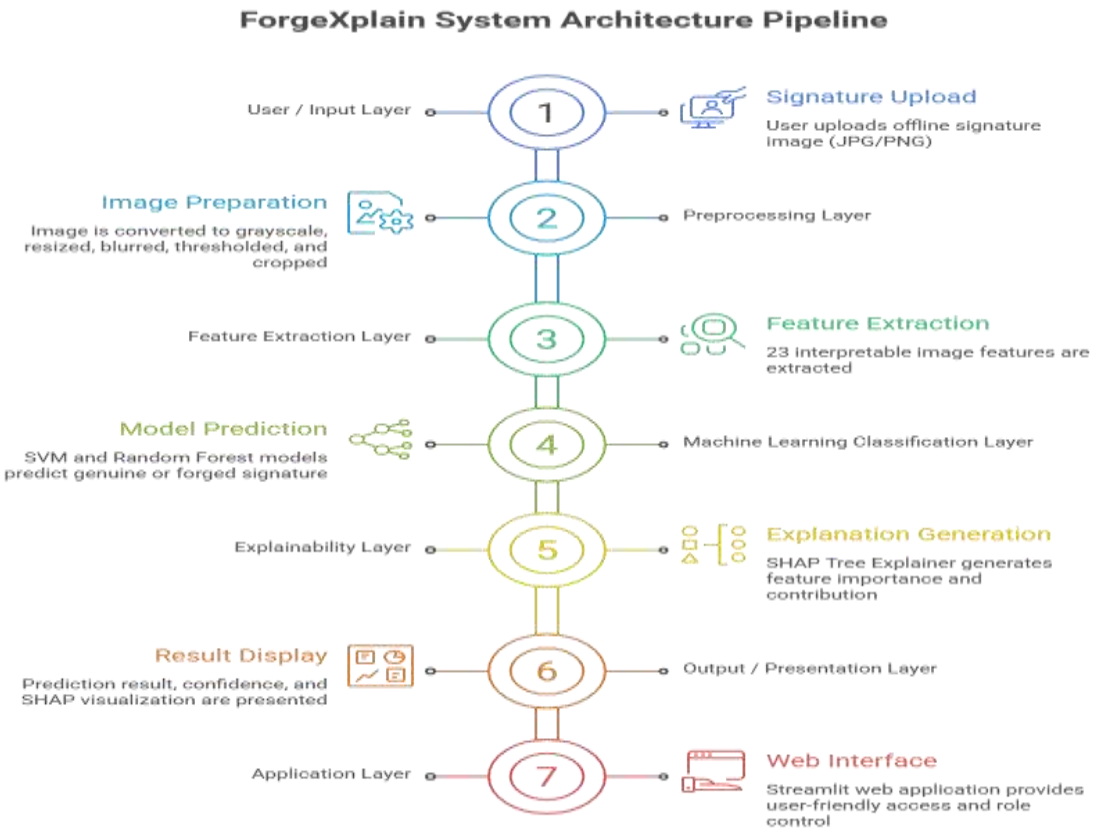


Figure 1.1 System Architecture

Advantages

• **Automated Signature Classification:** Reduces the effort required for manual examination.

• **Consistent Image Preprocessing:** Applies the same preprocessing pipeline to signature images.

• **Interpretable Feature Extraction:** Uses meaningful signature characteristics instead of relying only on raw images.

• **Multiple Machine Learning Models:** Supports comparison between SVM and Random Forest.

• **Explainable Predictions:** Uses SHAP to show the contribution of individual features.

• **User-Friendly Interface:** Provides signature analysis through a Streamlit web application.

• **Integrated Workflow:** Combines preprocessing, feature extraction, classification, and explanation into one framework.

Block diagram:

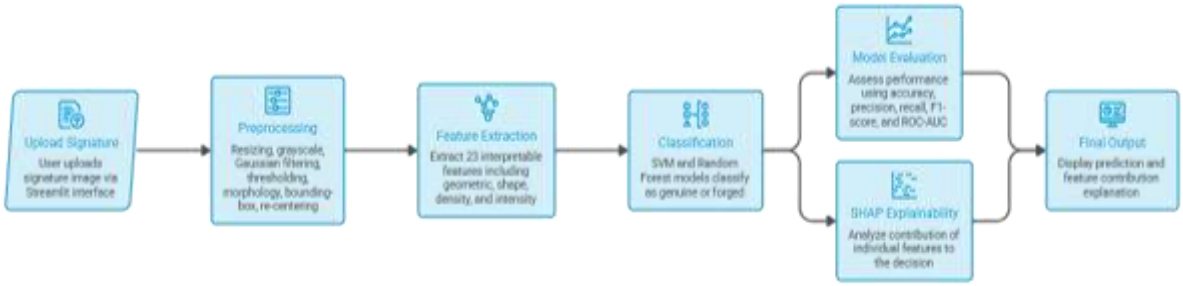


Figure 1.2 Workflow Diagram

1.8 Outcome of the Project

The main outcome of the project is a working web-based framework for offline signature forgery detection. The system accepts a signature image, performs preprocessing, extracts interpretable features, applies machine learning classification, and provides an explanation of the prediction.

The framework demonstrates how machine learning and Explainable AI can be combined for signature verification. Instead of limiting the output to a genuine or forged classification, the system provides feature-level information that helps users understand the factors influencing the prediction.

The developed Streamlit application provides a simple interface for interacting with the complete workflow. Thus, the project provides an integrated approach to automated and interpretable offline signature forgery detection.

1.9 Report Organization

Chapter 1: Introduction

This chapter introduces the concept of offline signature verification and explains the motivation, problem identification, scope, objectives, methodology, existing system, proposed system, and outcome of the project.

Chapter 2: Literature Survey

This chapter discusses existing research and related approaches in offline signature verification, machine learning-based forgery detection, image processing, and Explainable AI. It also identifies the limitations of existing approaches and establishes the need for the proposed system.

Chapter 3: System Analysis

This chapter presents the system analysis and feasibility study of the proposed project. It discusses technical, economic, operational, and other relevant feasibility aspects along with the requirements of the system.

Chapter 4: Requirement Analysis

This chapter describes the hardware and software requirements necessary for implementing and running the ForgeXplain system. It also presents the functional and non-functional requirements.

Chapter 5: System Design

This chapter explains the architecture and design of the proposed system. It presents the different modules, workflow, data processing stages, and design diagrams used for developing the system.

Chapter 6: Implementation

This chapter describes the implementation of the preprocessing pipeline, feature extraction, machine learning models, SHAP-based explanation, and Streamlit web application.

Chapter 7: Testing and Results

This chapter presents the testing process and evaluates the performance of the developed system. It discusses the classification results and evaluation metrics obtained from the machine learning models.

Chapter 8: Screenshots and Output

This chapter presents screenshots of the developed ForgeXplain application and demonstrates the different stages of input processing, prediction, explanation, and final output.

1.10 Introduction Summary

Offline signature verification distinguishes genuine signatures from forged ones, but manual and conventional automated methods face challenges related to time, image quality, signature variations, and interpretability.

ForgeXplain addresses these issues through image preprocessing, feature extraction, and SVM and Random Forest classifiers using 23 features. SHAP explains feature contributions, while a Streamlit interface integrates the process into a single application, enabling automated and explainable signature forgery detection.

ForgeXplain improves transparency by helping users understand the factors influencing each classification.

CHAPTER 2

LITERATURE SURVEY

2.1 Related Work

Offline signature verification has been an active research area because handwritten signatures are widely used for authentication in banking, financial, legal, and administrative applications. Research in this field has progressed from traditional image-processing and manually designed features toward machine learning and deep learning approaches. Recent studies have also focused on improving the interpretability of machine learning predictions so that automated signature verification systems can provide more meaningful decision support.

The literature related to ForgeXplain mainly covers five important areas: offline signature verification, signature image preprocessing, feature extraction, machine learning-based classification, and Explainable Artificial Intelligence. These studies provide the foundation for developing an integrated framework capable of detecting forged signatures while also explaining the features influencing the classification.

2.1.1 Reference Papers

Research papers on offline signature verification provide different approaches for distinguishing genuine signatures from forged signatures. Existing studies have investigated handcrafted features, statistical methods, Support Vector Machines, neural networks, Convolutional Neural Networks, and more recent explainable AI techniques.

A systematic review by Hameed et al. examined machine-learning-based offline signature verification systems and analyzed datasets, preprocessing techniques, feature extraction methods, classification models, and evaluation metrics. The review considered 56 studies and highlighted commonly used public datasets such as CEDAR, GPDS, MCYT-75, UTSig, and BHSig260.

These studies demonstrate the importance of preprocessing and feature representation before classification. They also show the increasing use of machine learning and deep learning to improve the automation of signature verification. ForgeXplain builds on these concepts by using a structured preprocessing pipeline, interpretable features, SVM and Random Forest classifiers, and SHAP-based explanations.

2.1.2 Traditional Feature-Based Signature Verification

Traditional offline signature verification approaches generally depend on extracting visual and statistical characteristics from signature images. Features may describe the geometry, shape, texture, density, contour, and intensity distribution of a signature.

Such approaches have the advantage of producing features that can be examined directly. However, their performance depends strongly on selecting appropriate features and handling variations between genuine signatures produced by the same writer.

The systematic review by Hameed et al. specifically identifies preprocessing and feature extraction as major components of offline signature verification systems. It also reports that different machine learning models have been investigated using several public signature datasets.

ForgeXplain follows this feature-based direction by extracting 23 interpretable features covering geometric, shape, density, contour, and intensity-histogram characteristics.

2.1.3 Machine Learning-Based Offline Signature Verification

Machine learning provides a method for learning the difference between genuine and forged signatures from labelled examples. Among the commonly used classification approaches, Support Vector Machines have been widely investigated for signature verification because they can perform classification using feature vectors derived from signature images.

Research on offline signature verification has also investigated neural networks and deep learning. A study on recurrent neural-network-based offline signature verification evaluated multiple public datasets, including CEDAR, GPDS, and MCYT-75, demonstrating the continued development of learning-based verification systems.

Another important direction is feature learning using deep convolutional neural networks. Hafemann et al. investigated learned representations for offline handwritten signature verification and evaluated them using datasets including GPDS, MCYT-75, CEDAR, and PUC-PR.

ForgeXplain uses a comparatively interpretable machine-learning approach by combining manually defined but meaningful features with **SVM and Random Forest** classification rather than depending only on an end-to-end deep neural network.

2.2 Machine Learning-Based Offline Signature Verification Systems

Hameed et al. presented a systematic review of machine-learning-based offline signature verification systems. The study analyzed research published between 2014 and 2019 and examined five major aspects: datasets, preprocessing, feature extraction, machine-learning verification models, and performance metrics.

The review covered public datasets including CEDAR, GPDS, MCYT-75, UTSig, and BHSig260. It showed that preprocessing and feature extraction play an important role in the performance of offline signature verification systems. The review also identified several open research challenges, including improving robustness and generalization.

2.3 Deep Learning for Offline Handwritten Signature Verification

NAME	YEAR	AUTHOR	FEATURE	ADVANTAGE	DISADVANTAGE
Machine learning-based offline signature verification systems: A systematic review	2021	M. Muzaffar Hameed et al.	Reviews datasets, preprocessing, feature extraction, ML models and evaluation methods	Provides a broad analysis of existing ML-based offline signature verification research	Identifies continuing issues related to robustness, accuracy and generalization

Hafemann, Sabourin, and Oliveira investigated the use of deep convolutional neural networks for learning features for offline handwritten signature verification. Instead of relying entirely on manually designed features, the approach learns feature representations from signature images.

The work evaluated the learned representations using several signature datasets, including CEDAR, GPDS, MCYT-75, and PUC-PR. The study demonstrated the potential of deep learning to automatically learn useful signature representations for writer-dependent verification.

Deep learning reduces the need for manually designing every feature, but it can also require greater computational resources and may be more difficult to interpret than traditional feature-based models. ForgeXplain therefore uses interpretable features so that the contribution of individual characteristics can be presented to the user.

NAME	YEAR	AUTHOR	FEATURE	ADVANTAGE	DISADVANTAGE
Learning features for offline handwritten signature verification using deep convolutional neural networks	2017	L. G. Hafemann, R. Sabourin, L. S. Oliveira	Learns signature representations using deep CNN features	Reduces dependence on manually designed features	Deep models can require significant computational resources and are less directly interpretable

2.4 Offline Signature Verification Using Region-Based Deep Metric Learning

Research has also explored deep metric learning for offline signature verification. One such approach uses local regions of signature images together with a deep feature extraction network. The method evaluates relationships between signature representations and combines information from different regions to make a verification decision.

The approach was evaluated using public datasets including CEDAR and GPDS Synthetic, along with a Chinese signature dataset. Such methods demonstrate that both global and local characteristics of a signature can be useful for verification.

The main limitation of highly specialized deep architectures is their complexity. In comparison, ForgeXplain uses a smaller interpretable feature representation, making it easier to inspect which signature characteristics influence the final classification.

NAME	YEAR	AUTHOR	FEATURE	ADVANTAGE	DISADVANTAGE
Offline signature verification using a region based deep metric learning network	2021	Research study on region-based deep metric learning	Uses global and local signature regions for deep feature representation	Captures detailed local characteristics of signatures	Deep architecture is more complex and less directly interpretable

2.5 CEDAR Dataset and Signature-Based Classification

The **CEDAR Signature Dataset** is one of the commonly used public datasets in offline signature verification research. It contains signatures from 55 writers, with 24 genuine and 24 forged signatures for each writer, giving 1,320 genuine and 1,320 forged signatures.

Because the dataset contains both genuine and skilled forged signatures, it provides a useful benchmark for evaluating signature forgery detection approaches. Several studies use CEDAR to compare preprocessing techniques, feature representations, machine learning models, and deep learning approaches.

ForgeXplain also uses the CEDAR dataset for developing and evaluating its signature classification framework. The dataset therefore provides a common basis for comparing the proposed feature-based machine learning approach with existing research.

NAME	YEAR	AUTHOR	FEATURE	ADVANTAGE	DISADVANTAGE
CEDAR Signature Dataset	2004	CEDAR research group	Genuine and forged offline signature images from 55 writers	Widely used benchmark with genuine and skilled forged signatures	Limited number of writers compared with some newer large-scale datasets

2.6 Hybrid Machine Learning Framework for Offline Signature Verification

Recent research has investigated hybrid machine-learning frameworks that combine optimization techniques with handcrafted image features. A recent study introduced a framework using Gray Wolf Optimization together with texture-based feature extraction and evaluated it on datasets including CEDAR, SID, and BHSig260.

The study uses image preprocessing followed by feature extraction based on local binary pattern variants. On CEDAR, the reported framework achieved high verification performance, demonstrating the continuing relevance of carefully designed image features and optimization techniques in offline signature verification.

Such approaches can achieve strong classification performance but may involve additional optimization stages and more complex processing. ForgeXplain instead emphasizes a straightforward pipeline consisting of preprocessing, interpretable feature extraction, classification, and explanation.

NAME	YEAR	AUTHOR	FEATURE	ADVANTAGE	DISADVANTAGE
A hybrid machine learning framework for offline signature verification using gray wolf optimization	2026	N. C. R. et al.	GWO-based processing with LBP variants and feature-based verification	Combines optimization and texture features for strong verification performance	Additional optimization stages increase system complexity

2.7 Explainable AI in Signature Forgery Detection

While conventional machine learning and deep learning systems focus mainly on improving classification performance, explainability is increasingly important for applications such as forensic document analysis.

A recent study specifically investigated Explainable AI in signature forgery detection using deep learning architectures and several XAI techniques, including Grad-CAM, LayerGradCAM, Integrated Gradients, Saliency Maps, DeepLIFT, and SHAP. The study used the CEDAR dataset containing 1,320 genuine and 1,320 forged signatures and evaluated whether model explanations aligned with regions identified by a forensic handwriting examiner.

The research demonstrates the importance of providing explanations rather than treating the classifier as a completely black-box system. It also shows that different XAI methods can provide different levels of faithfulness and agreement with expert interpretation.

ForgeXplain applies this principle using **SHAP-based explanations** for its machine learning predictions. Instead of showing only whether a signature is classified as genuine or forged, the framework provides feature-level information indicating how individual extracted characteristics influence the prediction

NAME	YEAR	AUTHOR	FEATURE	ADVANTAGE	DISADVANTAGE
Explainable AI in signature forgery detection	2025	Mehmet Türkay Yoldar, Recep Eryiğit, Nergis Cantürk	Uses XAI methods including SHAP, Integrated Gradients and DeepLIFT	Improves interpretability and evaluates explanations against expert input	Advanced deep-learning and XAI methods can require considerable computation

2.8 Comparison of Existing Approaches with ForgeXplain

The reviewed literature shows that offline signature verification has progressed from manually designed image features to machine learning, deep learning, metric learning, and explainable AI.

Traditional feature-based approaches provide relatively interpretable representations but depend on careful feature selection. Deep learning approaches can automatically learn complex representations but generally require more computational resources and may be difficult to interpret. Recent XAI research addresses this interpretability problem by explaining model predictions.

ForgeXplain combines the advantages of these directions by using a structured image-processing pipeline, interpretable feature extraction, machine learning classification, and SHAP-based explanations in one framework.

APPROACH	MAIN TECHNIQUE	ADVANTAGE	LIMITATION
Traditional verification	Manual/image-based comparison	Simple and understandable	Time-consuming and subjective
Feature-based ML	Handcrafted features + classifier	Interpretable features	Feature selection is important
Deep Learning	CNN/deep feature learning	Learns complex representations automatically	Higher computational requirement
Metric learning	Deep similarity/feature learning	Captures complex signature relationships	More complex architecture
XAI-based verification	ML/DL + explanation	Provides insight into model decisions	Explanation quality depends on the XAI method
ForgeXplain	**Preprocessing + 23 interpretable features + SVM/RF + SHAP**	**Combines classification with feature-level explanation**	**Performance depends on the quality and representativeness of extracted features**

2.9 Literature Survey Summary

The literature survey shows that offline signature verification has evolved from traditional manual and handcrafted feature-based methods toward machine learning and deep learning approaches. Public datasets such as CEDAR have supported the evaluation of various verification techniques, including SVMs, CNNs, deep metric learning, and hybrid optimization methods.

The reviewed studies highlight that preprocessing and feature extraction remain important components of offline signature verification. Although deep learning can automatically learn complex representations, interpretable feature-based methods are valuable when understanding the factors influencing a prediction is important. Recent Explainable AI (XAI) techniques such as SHAP and Integrated Gradients further improve the transparency of automated verification systems.

Based on these observations, ForgeXplain combines image preprocessing, 23 interpretable signature features, SVM and Random Forest classifiers, and SHAP-based explanations into a single web-based framework. The system aims to provide automated forgery detection while also showing the features contributing to the model's prediction.

CHAPTER 3

SYSTEM ANALYSIS

3.1 Introduction to System Analysis

The system analysis for the proposed ForgeXplain: Explainable Machine Learning Framework for Offline Signature Forgery Detection involves a detailed study o

f the problem of offline signature verification, the requirements of the proposed system, the technologies used, and the interaction between different modules. The main purpose of system analysis is to understand how the proposed system can be developed to identify forged signatures while providing an understandable explanation of the machine learning prediction.

Offline signature verification is an important application of image processing and machine learning. Unlike online signature verification, where information such as pen movement, writing speed, pressure, and stroke sequence may be available, offline signature verification works only with a static image of the signature. This makes the problem more challenging because the system must analyze visual characteristics present in the signature image.

ForgeXplain is designed as an explainable machine learning framework that accepts an offline signature image as input and processes it through a sequence of image-processing and machine-learning stages. The system first performs preprocessing operations to improve the quality of the input image. These operations include resizing, grayscale conversion, Gaussian blurring, adaptive thresholding, morphological operations, bounding-box extraction, and re-centering of the signature.

After preprocessing, relevant features are extracted from the signature image. The proposed framework considers interpretable characteristics related to the geometry, shape, density, contour, intensity distribution, and histogram of the signature. These features are then supplied to machine learning models for classification.

The proposed system uses Support Vector Machine (SVM) and Random Forest (RF) algorithms for classifying signatures as genuine or forged. The performance of the models can be evaluated using measures such as accuracy, precision, recall, F1-score, and ROC-AUC.

A major feature of ForgeXplain is the integration of Explainable Artificial Intelligence (XAI). The system uses SHAP-based explanations to identify the contribution of individual features toward the model's prediction. This makes the system more transparent than a conventional classification system that only provides a final result.

The complete system is implemented through a Streamlit-based web interface. The interface allows users to upload a signature image and obtain the classification result along with an explanation of the prediction. Thus, the system combines image processing, feature engineering, machine learning, explainability, and web-based interaction into a single framework.

3.2 Feasibility Study

System feasibility analysis determines whether the proposed ForgeXplain framework can be successfully developed, implemented, and maintained using the available resources, technology, time, and user requirements. The feasibility study helps in identifying possible limitations before the actual implementation of the system.

The proposed system is analysed from different perspectives to determine its practicality and suitability. The major feasibility studies considered for ForgeXplain are technical feasibility, economic feasibility, operational feasibility, schedule feasibility, and social feasibility.

3.2.1 Technical Feasibility

The proposed ForgeXplain system is technically feasible because the required programming languages, libraries, frameworks, and machine learning technologies are readily available. The system can be developed using Python and commonly available open-source libraries.

Python is used as the primary programming language because it provides extensive support for image processing, data manipulation, machine learning, and explainable AI. OpenCV can be used for processing the signature images and performing operations such as resizing, grayscale conversion, blurring, thresholding, morphological operations, and region extraction.

The feature extraction stage can be implemented using Python-based numerical and image-processing libraries. The extracted features are structured into a suitable format and supplied to the machine learning models.

The classification module uses Support Vector Machine and Random Forest algorithms. These algorithms are supported by the Scikit-learn library and can be trained and evaluated using the signature dataset. Random Forest is particularly suitable for the proposed framework because it can work with multiple numerical features and can also be integrated with tree-based explainability techniques.

The explainability module uses SHAP (SHapley Additive exPlanations) to analyze the contribution of individual features to the model's output. This enables ForgeXplain to provide feature-level explanations rather than only displaying the final classification.

The web application is developed using Streamlit, which provides a simple method for deploying Python-based machine learning applications. It allows users to upload signature images, initiate processing, and view prediction results through a graphical interface.

The system does not require highly specialized hardware for basic development and testing. A standard computer with adequate RAM, storage, and processing capability can be used. Therefore, the required technical resources are available and the system is technically feasible.

3.2.2 Economic Feasibility

The proposed ForgeXplain system is economically feasible because it primarily uses open-source programming languages, libraries, and development frameworks. The major technologies used in the system do not require expensive commercial licenses.

Python, OpenCV, NumPy, Pandas, Scikit-learn, SHAP, and Streamlit are available for development without the need for costly software licenses. This significantly reduces the development cost of the project.

The system can also be developed using a standard computer without requiring a dedicated high-end workstation. Although additional computing resources may be useful for larger datasets or advanced experiments, they are not essential for the basic implementation of the proposed framework.

The use of an existing signature dataset also reduces the cost and effort required for collecting and manually preparing a large number of signature samples. The dataset can be organized into genuine and forged categories for training and evaluation.The deployment cost can also be kept low by using suitable web-hosting or application deployment services. For an academic implementation, the system can therefore be developed and demonstrated with limited financial resources.

Hence, considering the availability of free development tools, open-source libraries, existing datasets, and standard hardware requirements, ForgeXplain is economically feasible.

3.2.3 Operational Feasibility

Operational feasibility determines whether the proposed system can be easily operated by users and whether it can perform its intended functions in a practical environment.

ForgeXplain is designed with a simple web-based interface. The user does not need to have detailed knowledge of machine learning algorithms to operate the system. The main interaction involves uploading an offline signature image and viewing the resulting classification and explanation.

The general operation of the system is as follows:

1. The user opens the ForgeXplain web application.
2. The user uploads an offline signature image.
3. The system receives and validates the uploaded image.
4. The image undergoes preprocessing.
5. The relevant signature region is identified and processed.
6. Interpretable features are extracted from the processed signature.
7. The extracted features are provided to the trained machine learning model.
8. The model predicts whether the signature is genuine or forged.
9. The system displays the classification result.
10. SHAP-based explanation is generated to show the contribution of important features.
11. The user can analyse the prediction using the displayed explanation.

The system automates most of the processing steps, reducing the amount of manual analysis required. This makes the system suitable for demonstration and research purposes.

The explainability component is particularly important from an operational perspective. A traditional machine learning system may simply display a classification result without providing information about the reason behind the prediction. ForgeXplain attempts to address this limitation by displaying the contribution of individual features.

The system can therefore be operated through a relatively simple workflow. However, for deployment in real-world authentication environments, additional security, privacy, validation, and performance requirements would need to be considered.

3.2.4 Schedule Feasibility

Schedule feasibility determines whether the proposed project can be completed within the available academic project period. The development of ForgeXplain is divided into different stages so that each component can be developed, tested, and integrated systematically.

The first stage involves identifying the problem and defining the objectives of the project. This is followed by a literature survey of offline signature verification, machine learning classification, image preprocessing, feature extraction, and explainable AI techniques.

The next stage involves dataset preparation. The signature dataset is studied and organized into appropriate categories for genuine and forged signatures. The data is then prepared for training and testing.The preprocessing module is developed after dataset preparation. Different image-processing operations are implemented to improve signature images and obtain a suitable representation for feature extraction.

The feature extraction stage follows preprocessing. Relevant interpretable features are extracted from the signature images and arranged in a suitable feature set.

The machine learning models are then trained using the extracted features. SVM and Random Forest models are evaluated to determine their performance for signature classification.

After model development, the explainability component is integrated. SHAP is used to identify the contribution of different features to the model prediction.

The trained model and explainability module are then integrated into the Streamlit application. The interface is tested using different signature images to verify that the complete pipeline operates correctly.

The major stages of the project are:

- Problem identification and requirement analysis
- Literature survey
- Dataset collection and preparation
- Image preprocessing

- Feature extraction
- Machine learning model development
- Model evaluation
- Explainability integration
- Streamlit web application development
- System testing and validation
- Documentation and final presentation

Since the project is divided into clearly defined stages, the required activities can be completed systematically within the available project schedule. Therefore, the proposed system is schedule feasible.

3.2.5 Social Feasibility

Social feasibility determines whether the proposed system can be accepted and used effectively by users and whether it provides meaningful benefits without creating unnecessary social or privacy concerns.

Signature verification is commonly associated with identity authentication and document verification. An automated system capable of analysing offline signatures can provide an additional method for examining signatures and identifying potentially forged samples.

ForgeXplain provides an advantage by combining classification with explainability. Instead of providing only a genuine or forged prediction, the system provides information about the features that contributed to the prediction. This can improve the transparency of the machine learning system and help users understand the output.

The system can also be useful for educational and research purposes. It demonstrates the practical application of image processing, machine learning, feature engineering, and explainable artificial intelligence in a biometric-related problem.

However, signature images may contain sensitive information and therefore need to be handled responsibly. In a real-world implementation, appropriate measures would be required for secure storage, controlled access, data protection, and privacy.

The prediction produced by the system should also be considered as a decision-support output rather than an absolute determination. Machine learning predictions can be affected by image quality, variations in handwriting, dataset characteristics, and other factors.

Considering its potential applications, simple user interaction, explainability, and research value, the proposed system is socially feasible when implemented with appropriate privacy and responsible-use measures.

3.3 Functional Requirements

Functional requirements describe the specific functions that the ForgeXplain system must perform in order to achieve its intended objectives.

1. Signature Image Upload

- The system must allow the user to upload an offline signature image.
- The system should accept supported image formats for processing.
- The uploaded image should be passed to the preprocessing module.
- The system should provide an appropriate message if an invalid or unsupported image is uploaded.

2. Image Preprocessing

- The system must preprocess the uploaded signature image before feature extraction.
- It should perform grayscale conversion to simplify image representation.
- The image should be resized to a suitable dimension for consistent processing.
- Gaussian blurring can be applied to reduce unwanted noise.
- Adaptive thresholding should be used to separate the signature from the background.
- Morphological operations such as opening and closing should be applied where required.
- The system should identify the relevant signature region using bounding-box processing.
- The extracted signature should be re-centered for further analysis.

3. Signature Feature Extraction

- The system must extract relevant features from the processed signature image.
- The extracted features should represent different characteristics of the signature.
- The feature set should include interpretable characteristics related to shape, geometry, density, contour, intensity, and histogram information.
- The extracted features must be converted into a suitable numerical representation for machine learning.
- The same feature extraction process should be applied consistently to training and prediction data.

4. Machine Learning Classification

- The system must use trained machine learning models for signature classification.

- The proposed framework should support SVM and Random Forest models.
- The model should classify the input signature as genuine or forged.
- The system should load the trained model before performing prediction.
- The classification output should be displayed clearly to the user.

5. Explainable AI Module

- The system must provide an explanation of the machine learning prediction.
- SHAP-based explanations should be generated for the classification result.
- The system should identify important features contributing to the prediction.
- The explanation should be presented in an understandable form.
- The explanation should help the user understand why the model produced a particular prediction.

6. Web-Based User Interface

- The system must provide an interactive Stream lit-based interface.
- The interface should provide an option for uploading a signature image.
- It should display the processed signature or relevant intermediate information where required.
- It should display the final classification result.
- It should display the corresponding feature-based explanation.
- The interface should provide suitable messages during processing and in case of errors.

7. Model and Application Integration

- The system must integrate image preprocessing, feature extraction, classification, and explanation into a single workflow.
- The output from preprocessing must be correctly passed to the feature extraction module.
- The extracted features must be correctly supplied to the trained model.
- The prediction must be connected to the explainability module.
- The final output must be displayed through the web interface.

8. Error Handling

- The system should detect invalid image inputs.
- It should provide suitable error messages when processing fails.

- The application should prevent unexpected input from causing complete system failure.
- Appropriate handling should be provided for missing or unsuitable data.

3.4 Non-Functional Requirements

Non-functional requirements specify the quality characteristics and operational constraints that the ForgeXplain system should satisfy.

1. Performance Requirements

The system should process signature images within a reasonable amount of time. Image preprocessing, feature extraction, model prediction, and explanation generation should be performed efficiently.

- The system should provide a prediction without unnecessary processing delays.
- The preprocessing pipeline should be optimized for repeated use.
- The trained model should be loaded efficiently.
- The application should provide responsive interaction through the web interface.
- SHAP explanation generation should be performed efficiently for the selected model.

2. Accuracy and Reliability Requirements

The system should provide consistent predictions when the same input and processing conditions are used.

- The trained model should be evaluated using suitable performance metrics.
- The preprocessing pipeline should remain consistent between training and prediction.
- The system should minimize processing errors.
- The application should handle unsuitable input images appropriately.
- The classification and explanation modules should work together consistently.

3. Usability Requirements

The system interface should be simple and understandable for users.

- Users should be able to upload a signature image easily.
- The prediction result should be displayed clearly.
- The explanation should be presented in a readable format.
- The system should avoid unnecessary technical complexity in the user interface.
- Appropriate instructions and messages should be displayed where required.

4. Security Requirements

Since signature images may contain sensitive information, security and privacy should be considered.

- Uploaded signature images should be handled securely.
- Sensitive information should not be unnecessarily stored.
- Access to the application and its resources should be controlled where required.
- The system should prevent unauthorized access to stored models and data.
- Appropriate secure communication should be used for deployed applications.

5. Maintainability Requirements

The system should be designed so that individual components can be modified or improved without requiring a complete redesign.

- The preprocessing module should be independently maintainable.
- Feature extraction should be separated from model prediction.
- Machine learning models should be replaceable or upgradeable.
- The explainability module should be independently maintainable.
- The Streamlit interface should allow future improvements.

6. Scalability Requirements

The system should provide scope for future expansion.

- Additional machine learning algorithms can be incorporated.
- Additional signature features can be added.
- Different datasets can be used for further evaluation.
- Additional explainability techniques can be integrated.
- The application can be extended to support larger datasets and more users.

7. Portability Requirements

- The system should run on standard desktop or laptop computers.
- The application should support commonly used operating systems where the required dependencies are available.
- The Streamlit interface should be accessible through modern web browsers.

8. Availability Requirements

The deployed application should be available to users when required.

- The web application should remain accessible during normal operating conditions.
- The system should handle temporary processing errors appropriately.
- The application should provide meaningful messages if a service or component becomes unavailable.
- The system should be capable of being restarted without requiring complete reconfiguration.

3.5 System Analysis Summary

The system analysis of ForgeXplain: Explainable Machine Learning Framework for Offline Signature Forgery Detection provides an understanding of the proposed system, its feasibility, and its functional and non-functional requirements.

The system integrates image preprocessing, interpretable feature extraction, machine learning classification, and Explainable AI into a unified framework. The uploaded signature is preprocessed, relevant features are extracted, and the data is classified using SVM and Random Forest models.

The feasibility study confirms that the system is technically and economically feasible due to the availability of required programming languages, open-source libraries, machine learning algorithms, and deployment frameworks. The simple web interface also makes the system operationally feasible for users without detailed machine learning knowledge.

The project follows well-defined stages, including dataset preparation, preprocessing, feature extraction, model development, SHAP integration, application development, testing, and documentation. Social feasibility highlights its potential use in research and education while emphasizing privacy and responsible handling of signature data.

The functional requirements include image uploading, preprocessing, feature extraction, classification, SHAP-based explanation, web interaction, and error handling. Non-functional requirements focus on performance, reliability, usability, security, maintainability, scalability, portability, and availability.

Overall, the system analysis establishes a clear foundation for implementing ForgeXplain and supports the detailed Requirement Analysis and System Design discussed in the following chapters.

CHAPTER 4

REQUIREMENT ANALYSIS

4.1 Functional Requirements

Functional requirements describe the specific operations and behaviours that the ForgeXplain system must perform. These requirements define how the system accepts an offline signature image, processes the image, extracts relevant features, performs classification, and provides an explainable prediction to the user.

1. Signature Image Upload

- The system must allow the user to upload an offline signature image through the web interface.
- The system should accept commonly used image formats such as JPG, JPEG, and PNG.
- The uploaded image should be passed to the preprocessing module for further processing.
- The system should provide an appropriate message if an invalid or unsupported image is uploaded.

2. Image Preprocessing

- The system must preprocess the uploaded signature image before feature extraction.
- The image should be resized to a suitable and consistent dimension.
- The system should convert the input image into grayscale for further processing.
- Gaussian blurring should be applied where required to reduce unwanted noise.
- Adaptive thresholding should be used to separate the signature from the background.
- Morphological operations such as opening and closing should be applied to improve the signature representation.
- The system should identify the signature region using bounding-box processing.
- The extracted signature should be cropped and re-centered before feature extraction.

3. Signature Feature Extraction

- The system must extract relevant features from the pre-processed signature image.
- The features should represent important visual characteristics of the signature.
- The system should extract interpretable characteristics related to geometry, shape, density, contour, intensity, and histogram distribution.
- The extracted features should be converted into a numerical feature representation.

- The same feature extraction procedure must be followed for both training and prediction.

4. Machine Learning Classification

- The system must use trained machine learning models for signature classification.
- The proposed system should support **Support Vector Machine (SVM)** and **Random Forest (RF)** classifiers.
- The extracted signature features must be supplied to the selected trained model.
- The model should classify the signature as **genuine or forged**.
- The classification result should be displayed clearly on the user interface.

5. Explainable AI Module

- The system must provide an explanation for the generated classification result.
- The system should use SHAP-based explainability to analyze the model prediction.
- It should identify important features that contribute to the prediction.
- The contribution of individual features should be represented in an understandable form.
- The explanation should help the user understand the factors influencing the model's decision.

6. User Interface

- The system must provide a simple and interactive Streamlit-based web interface.
- The interface should provide an option for uploading the signature image.
- It should display appropriate processing information to the user.
- The final prediction should be clearly displayed.
- The feature-level explanation should be presented along with the prediction.
- The interface should be easy to understand and operate without requiring programming knowledge.

7. Model Integration

- The system must integrate the preprocessing, feature extraction, classification, and explainability modules.
- The output of the preprocessing module must be correctly passed to the feature extraction module.
- The extracted features must be supplied to the trained machine learning model.

- The classification result must be connected with the SHAP explanation module.
- The complete processing pipeline should execute without unnecessary manual intervention.

8. Error Handling

- The system should identify invalid or unsuitable input images.
- It should display suitable error messages when an operation cannot be completed.
- The application should handle missing input appropriately.
- Errors occurring during image processing or prediction should not cause the entire application to terminate unexpectedly.

9. Model Performance Evaluation

- The system should support evaluation of the developed classification models.
- Performance should be evaluated using suitable measures such as accuracy, precision, recall, F1-score, and ROC-AUC.
- The performance of SVM and Random Forest models can be compared during development.
- The evaluated model should be integrated into the final application based on the project implementation.

10. System Integration and Processing

- The application must integrate all major modules into one continuous workflow.
- The system should process the uploaded signature from image input to final prediction.
- The extracted information should be passed correctly between the different modules.
- The final result and explanation should be displayed through the web interface.

4.2 Non-Functional Requirements

Non-functional requirements define the performance, usability, reliability, security, and quality standards that the ForgeXplain system should satisfy.

1. Performance Requirements

The system should process uploaded signature images efficiently and provide the prediction without unnecessary delay.

- Image preprocessing should be performed efficiently.
- Feature extraction should require reasonable processing time.

- SHAP explanations should be generated within a reasonable processing time.
- The web interface should remain responsive during normal operation.

2. Reliability Requirements

Reliability ensures that the system performs consistently under normal operating conditions.

- The system should provide consistent results for the same input and processing conditions.
- The application should handle invalid inputs without crashing.
- Errors during processing should be handled appropriately.
- The complete pipeline should operate reliably from image upload to final prediction.
- The trained models and required processing components should be loaded correctly.

3. Accuracy Requirements

The classification system should provide reliable predictions based on the trained machine learning models.

- The models should be trained using appropriately prepared signature data.
- The system should use the same preprocessing and feature extraction procedure during training and prediction.
- Model performance should be evaluated using appropriate classification metrics.
- The system should provide explanations corresponding to the generated prediction.

4. Usability Requirements

The system interface should be simple, clear, and easy to use.

- Users should be able to upload an image without difficulty.
- The interface should provide clear instructions where required.
- The genuine or forged result should be displayed prominently.
- The explanation should be presented in an understandable manner.
- The system should not require users to have technical knowledge of machine learning.

5. Security Requirements

Signature images may contain sensitive information and therefore must be handled carefully.

- Uploaded data should be processed securely.
- Sensitive signature information should not be stored unnecessarily.

- Access to application resources and trained models should be controlled where required.
- The deployed web application should use secure communication mechanisms.
- Temporary data should be handled according to the privacy requirements of the deployment environment.

6. Maintainability Requirements

The system should be easy to modify, maintain, and improve.

- The preprocessing module should be independently maintainable.
- Feature extraction should be separated from classification.
- Machine learning models should be replaceable without redesigning the complete application.
- The explainability module should be independently maintainable.
- The source code should be organized and documented properly.

7. Scalability Requirements

The system should provide the possibility of future expansion.

- Additional machine learning models can be incorporated.
- Additional signature features can be added.
- Larger datasets can be used for future training and evaluation.
- Additional explainability techniques can be integrated.
- The system can be extended to support additional applications of offline signature verification.

8. Portability Requirements

The application should be capable of running on commonly used computing platforms.

- The Python-based system should run on supported versions of Windows, Linux, and macOS.
- The Streamlit web interface should be accessible through commonly used modern browsers.
- Required Python libraries and dependencies should be installable on supported platforms.
- The system should maintain consistent functionality across supported environments.

4.3 Software Requirements

The software environment and libraries required for the development and deployment of the ForgeXplain system include the following:

Category	Software/Library Used	Purpose
Programming Language	Python	Development of preprocessing, feature extraction, ML and application logic
Web Framework	Streamlit	Development and deployment of the interactive web application
Image Processing	OpenCV	Image preprocessing and signature-region processing
Numerical Computing	NumPy	Numerical operations and feature processing
Data Processing	Pandas	Dataset handling and data manipulation
Machine Learning	Scikit-learn	SVM, Random Forest and model evaluation
Explainable AI	SHAP	Generation of feature-level model explanations
Development Environment	Visual Studio Code / PyCharm	Coding, debugging and project development
Dataset	CEDAR Signature Dataset	Training and evaluation of signature classification models
Web Browser	Google Chrome / Microsoft Edge / Firefox	Accessing the Streamlit application

The above software components provide the necessary environment for developing the complete ForgeXplain framework. Python acts as the main programming language, while OpenCV and NumPy support image processing and numerical operations.

Scikit-learn provides the machine learning algorithms required for classification, while SHAP is used to provide explanations for model predictions. Streamlit provides the interface through which the different components are made available to the user.

4.3.1 Operating System

The operating system plays an important role in ensuring the smooth functioning of the ForgeXplain system. It manages system resources, provides the environment required for Python execution, and supports the different libraries and tools used during development and deployment.

Since the proposed system integrates image processing, machine learning, explainability, and web application components, the operating system must provide adequate support for Python libraries and browser-based interaction.

The system can be developed and operated on commonly used operating systems such as **Windows, Linux, and macOS**.

a. Windows 10 / Windows 11 (64-bit)

Purpose: Primary development and testing platform.

Advantages:

- Provides broad compatibility with Python and machine learning libraries.
- Supports development tools such as Visual Studio Code and PyCharm.
- Provides easy access to web browsers for testing the Streamlit interface.
- Suitable for OpenCV-based image processing and Scikit-learn model execution.
- Provides a user-friendly environment for academic project development.

Minimum Requirements:

- Processor: Intel Core i3 / AMD Ryzen 3 or higher
- RAM: 8 GB recommended
- Storage: At least 10 GB free space
- Network: Stable internet connection when required for deployment or package installation
- Browser: Google Chrome, Microsoft Edge, or Firefox

b. Linux

Purpose: Suitable for development, testing, and server-side deployment.

Advantages:

- Provides a stable environment for Python-based applications.
- Supports open-source machine learning and image-processing frameworks.

- Suitable for local and cloud-based deployment.
- Provides efficient resource management for application execution.

Minimum Requirements:

- Processor: Intel Core i3 / AMD equivalent or higher
- RAM: 8 GB recommended
- Storage: At least 10 GB free space
- Network: Required when accessing remote services or deploying the application
- Browser: A supported modern web browser

c. macOS

Purpose: Optional platform for development and academic use.

Advantages:

- Supports Python-based development environments.
- Provides compatibility with OpenCV, NumPy, Pandas, Scikit-learn, and SHAP.
- Suitable for running and testing Streamlit applications.
- Provides a stable environment for software development.

Limitations:

- Some library configurations may differ from Windows or Linux.
- Additional configuration may occasionally be required for specific machine learning environments.

4.4 Software Requirement Summary

The ForgeXplain system integrates multiple software technologies to perform offline signature forgery detection and provide explainable machine learning predictions. The software requirements define the programming language, libraries, frameworks, development environment, dataset, and browser required for implementing and operating the system.Python is used as the primary programming language because of its extensive support for image processing, data analysis, machine learning, and explainable AI. It provides a common environment in which the different modules of the proposed system can be integrated.OpenCV is used for processing signature images. Operations such as grayscale conversion, resizing, Gaussian blurring, thresholding, morphological processing, bounding-box extraction, and re-centering can be performed using image-processing functions.

Scikit-learn provides the machine learning algorithms used for classification. In ForgeXplain, SVM and Random Forest are used to analyze the extracted signature features and classify the input as genuine or forged.

SHAP provides the explainability component of the system. It helps identify the contribution of individual features toward the model prediction and allows the system to provide a more transparent output.

Streamlit is used to create the web-based interface. It connects the user interface with the underlying Python-based processing pipeline and allows the user to upload signature images and view the classification and explanation.

The combination of these software components provides a complete environment for implementing the ForgeXplain framework.

4.5 Hardware Requirements

The ForgeXplain system mainly performs image processing, feature extraction, machine learning prediction, and web-based interaction. Therefore, a stable computing environment with sufficient processing power and memory is required.

The hardware requirements can be divided into minimum and recommended configurations depending on the intended use of the system.

Component	**Specification**	**Purpose / Description**
Processor (CPU)	Intel Core i3 / AMD Ryzen 3 or higher	Handles image processing, feature extraction and model execution
Memory (RAM)	8 GB minimum	Supports Python, libraries, browser and application execution
Storage	10 GB or more free space	Required for software, datasets, models and temporary files
Graphics	Integrated GPU sufficient for basic operation	Supports normal display and application execution
Display	1366 × 768 resolution or higher	Displays the Streamlit interface and prediction results
Keyboard	Standard keyboard	Used for normal computer interaction
Mouse / Touchpad	Standard pointing device	Used for uploading images and navigating the interface

Network	Stable internet connection	Required for package installation and web deployment/access
Camera	Not mandatory	The system works with uploaded offline signature images

The proposed system does not require a dedicated GPU for normal model prediction when using the selected traditional machine learning algorithms. A standard laptop or desktop computer can therefore be used for development, testing, and demonstration.

For larger-scale model training or future extensions involving deep learning, additional processing resources may be required.

4.6 Hardware Requirements Summary

The above hardware specifications represent the basic configuration required for developing, testing, and demonstrating the ForgeXplain system.

An 8 GB RAM system with a suitable modern processor is sufficient for running the Python environment, image-processing libraries, machine learning models, SHAP, and Streamlit application under normal project conditions. Adequate storage is required for maintaining the dataset, trained models, project files, and required software libraries.

A dedicated graphics processing unit is not essential for the current implementation because the proposed framework primarily uses traditional machine learning algorithms such as SVM and Random Forest. However, additional computing resources may be useful if the project is extended to deep-learning-based signature verification in the future.

The system therefore does not require specialized hardware and can be implemented using commonly available laptop or desktop configurations.

4.7 Requirement Analysis Summary

This chapter describes the major software and hardware requirements necessary to implement the ForgeXplain: Explainable Machine Learning Framework for Offline Signature Forgery Detection system.

The functional requirements define the major operations of the system, including signature image upload, preprocessing, feature extraction, machine learning classification, SHAP-based explanation, web-based interaction, error handling, and model evaluation.

The non-functional requirements define the quality characteristics of the system, including performance, reliability, accuracy, usability, security, maintainability, scalability, and portability. These requirements ensure that the system is not only capable of performing its main functions but can also be operated and maintained effectively.

The software requirement analysis identifies Python, OpenCV, NumPy, Pandas, Scikit-learn, SHAP, Streamlit, and other supporting tools as the major software components. The hardware analysis shows that a standard computer with adequate RAM, storage, and processing capability is sufficient for the current implementation.

Overall, the requirement analysis provides a clear foundation for the development of the ForgeXplain system. The identified requirements will be used in the following chapter to design the system architecture, workflow, modules, data flow, and other design components of the proposed framework.

CHAPTER 5

SYSTEM DESIGN

The System Design of ForgeXplain: Explainable Machine Learning Framework for Offline Signature Forgery Detection describes the architecture, modules, data flow, and interaction between the different components of the proposed system. It explains how a scanned signature image is processed, how important features are extracted, how machine learning models classify the signature, and how Explainable Artificial Intelligence (XAI) helps users understand the prediction.

The proposed system follows a modular architecture consisting of image preprocessing, feature extraction, machine learning classification, SHAP-based explanation, and a Streamlit-based user interface. Each module performs a specific task and communicates with the other modules to complete the verification process.

The system accepts scanned signature images or signature regions extracted from larger documents. The images are preprocessed to reduce noise and improve consistency. A fixed set of 23 interpretable features is extracted from each processed signature. The extracted features are supplied to Support Vector Machine (SVM) and Random Forest classifiers to distinguish genuine signatures from forged signatures. The system then displays the classification result and feature-level explanations through the web interface.

The design aims to provide a user-friendly, maintainable, and interpretable signature verification system. The integration of machine learning and SHAP-based explanations enables the user to examine the factors influencing a prediction rather than relying only on the final classification label.

5.1 System Architecture Overview

The overall architecture of ForgeXplain consists of five major functional modules.

1. User Interface Module

The user interface is developed using Streamlit and acts as the main point of interaction between the user and the system.

- Allows users to upload scanned signature images.
- Provides the interface for initiating signature verification.
- Supports confirmation or adjustment of a candidate signature region when required.
- Displays the classification result and available confidence information.
- Presents the SHAP-based explanation of the model's prediction.

2. Image Preprocessing and Segmentation Module

This module prepares the uploaded signature image for feature extraction.

- Resizes the image and converts it into grayscale format.
- Applies Gaussian blur to reduce noise.
- Uses adaptive thresholding to separate the signature strokes from the background.
- Applies morphological operations to improve the binary image.
- Crops and recenters the signature for consistent processing.
- Identifies candidate signature regions in larger documents for user confirmation or adjustment.

3. Feature Extraction Module

The feature extraction module converts the preprocessed signature into a numerical representation.

- Extracts 23 interpretable features from each signature image.
- Includes geometric, shape, density, contour, and intensity-related characteristics.
- Uses histogram-based information and Hu moments to represent additional image characteristics.
- Produces a fixed-length numerical feature vector for classification.

4. Machine Learning Classification Module

This module uses trained machine learning models to predict whether a signature is genuine or forged.

- Uses Support Vector Machine (SVM) and Random Forest classifiers.
- Accepts the numerical feature vector generated by the feature extraction module.
- Produces the predicted signature class.
- Provides associated confidence information where supported by the model implementation.
- Allows the performance of the classifiers to be evaluated and compared.

5. Explainable Artificial Intelligence Module

This module provides explanations for the model's predictions using SHAP (SHapley Additive exPlanations).

- Identifies features that influence individual predictions.
- Indicates the positive or negative contribution of features to the model output.
- Provides feature-level information to help users understand the classification.
- Presents explanations through the Streamlit interface.

The modules work together as a unified processing pipeline. The user uploads a signature image, the image is preprocessed, the relevant features are extracted, and the trained classifier generates a prediction. The explanation module then provides feature-level information associated with the prediction, and the final result is displayed to the user.

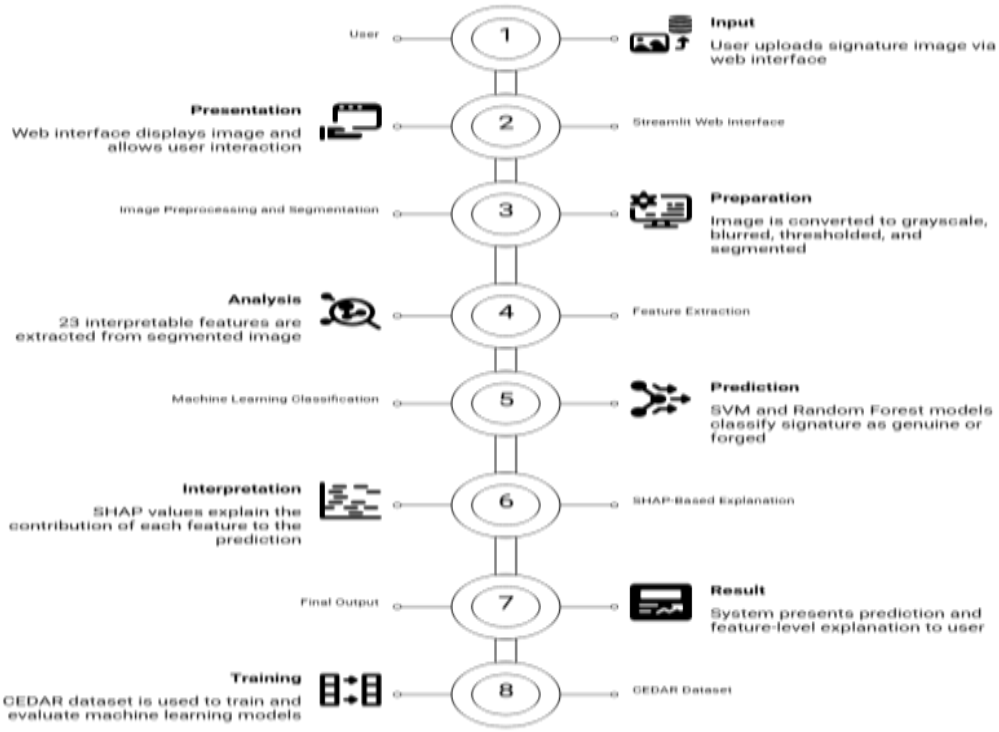


Figure 5.1: System Architecture of ForgeXplain

5.1.1 System Flow Design

The system flow design describes the sequence of operations performed by ForgeXplain, beginning with image upload and ending with the display of the prediction and explanation.

Step 1: User Input

The user opens the ForgeXplain web application and uploads a scanned signature image. If the signature is contained within a larger document, the system identifies a candidate region that can be confirmed or adjusted by the user.

Step 2: Image Preprocessing

The uploaded image is resized and converted into grayscale. Gaussian blur is applied to reduce noise, followed by adaptive thresholding to separate the signature strokes from the background. Morphological operations are then applied, and the signature is cropped and recentered.

Step 3: Feature Extraction

The preprocessed signature is analysed to extract 23 numerical features. These features represent geometric, shape, density, contour, and intensity characteristics, including histogram-based features and Hu moments.

Step 3: Feature Vector Generation

The extracted features are arranged into a fixed-length numerical feature vector. This vector becomes the input for the trained machine learning classifier.

Step 5: Signature Classification

The system uses the selected trained SVM or Random Forest classifier to predict whether the signature is genuine or forged. The prediction and supported confidence information are prepared for presentation.

Step 6: SHAP-Based Explanation

The explainability module analyses the contribution of individual features to the model's prediction. The SHAP values help indicate which features influenced the prediction and the direction of their contributions.

Step 7: Result Display

The Streamlit interface displays the predicted class and the available explanation information. The user can examine the output to understand the factors that influenced the model's decision.

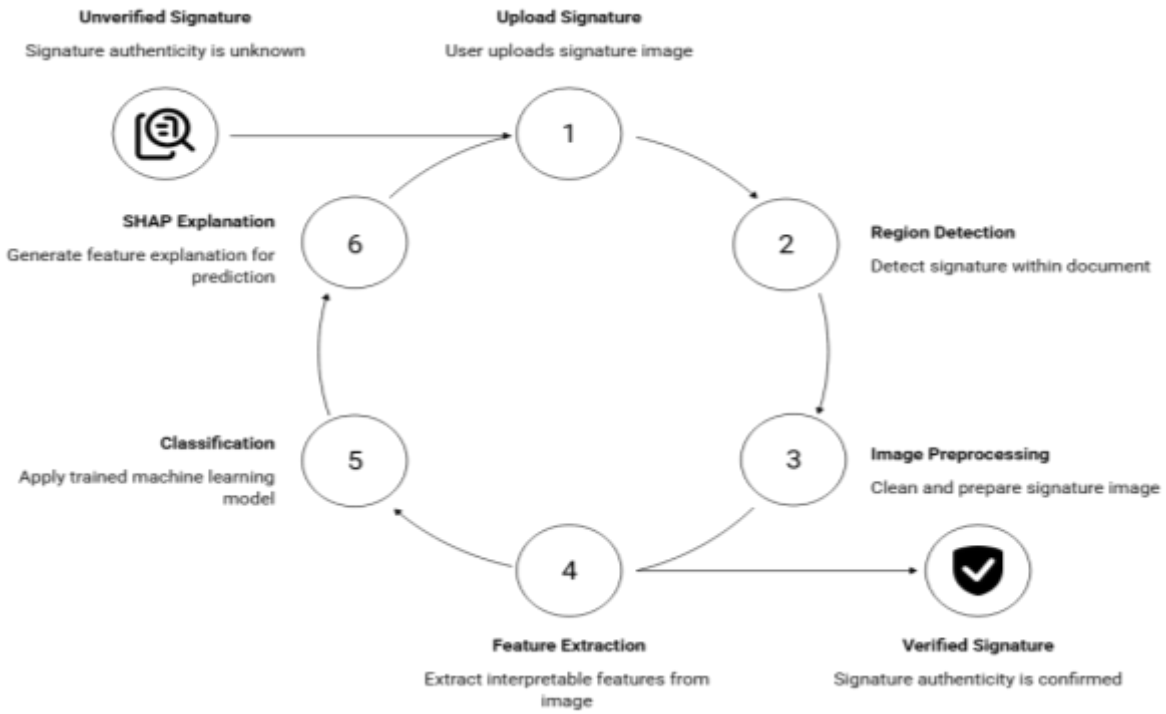


Figure 5.2: System Flow Design of ForgeXplain

5.1.2 Design Characteristics

The design of ForgeXplain follows several characteristics that support the functionality and maintainability of the system.

1. Modularity

The system is divided into separate modules for preprocessing, feature extraction, classification, explainability, and interface management. Each module performs a defined task, making the system easier to understand, test, and maintain.

2. Interpretability

ForgeXplain uses 23 interpretable features and SHAP-based explanations. This helps users examine the characteristics that influence a prediction instead of relying only on a genuine or forged label.

3. Consistent Image Processing

The preprocessing module applies a defined sequence of image operations. Resizing, grayscale conversion, denoising, thresholding, morphological processing, cropping, and recentering help produce consistent inputs for feature extraction.

4. Machine Learning Integration

The system integrates SVM and Random Forest classifiers into the same verification workflow. Their performance can be compared using suitable classification metrics.

5. User-Friendly Interface

The Streamlit interface provides a simple method for uploading signature images and viewing results. Users can access the principal functions without directly interacting with the underlying machine learning code.

6. Maintainability

The separation of system functions allows individual components to be modified or improved without redesigning the complete application, provided that the interfaces between components remain compatible.

7. Security and Access Control

The implementation incorporates role-based access control to separate ordinary verification tasks from restricted administrative functions, such as model management and retraining.

8. Decision Support

The system combines the classification result with feature-level explanations to support the review of signature verification decisions. The explanations provide additional information but do not independently prove that a signature is genuine or forged.

5.2 Gantt Chart

A Gantt chart represents the schedule of project activities and shows how the major development tasks are distributed over time. In ForgeXplain, the development activities can be divided into two phases: Phase One, which focuses on problem analysis and system design, and Phase Two, which focuses on implementation, integration, and testing.

The activities are arranged according to their dependencies. Requirement analysis and system design are completed before the main implementation work, while integration and testing are carried out after the individual modules have been developed.

5.2.1 Gantt Chart of Phase One

Phase One focuses on understanding the problem, studying existing approaches, identifying the project requirements, and designing the proposed system.

The major activities are:

1. Problem identification and definition of project objectives.
2. Literature survey of offline signature verification and forgery detection.
3. Study of the CEDAR dataset and machine learning approaches.
4. Identification of functional and non-functional requirements.
5. Selection of software tools, libraries, and algorithms.
6. Preparation of the system architecture.

Design of the system workflow and processing modules.

Sl. No.	Activity	Expected Outcome
1	Problem identification	Defined project objectives
2	Literature survey	Understanding of existing methods
3	Dataset study	Understanding of the CEDAR dataset
4	Requirement analysis	Identified system requirements
5	Technology selection	Selected tools and algorithms
6	System architecture design	Planned system components
7	Workflow design	Defined processing sequence

Table 5.1: Phase One Project Activities

	Feb 3	Feb 9	Feb 23	Apr 8	May 12	Jun 10	July 7	Aug 13	Sept 7	Sept 18	Oct 3	Oct 7
Planning phase	☑	☑										
Literature Survey			☑									
Analysis phase				☑								
Design phase												
Implementation												
Testing												
Deployment												
Documentation report												

Figure 5.3: Gantt Chart of Phase One

5.2.2 Gantt Chart of Phase Two

Phase Two focuses on implementing the system, integrating the different modules, and evaluating the completed application.

The major activities are:

1. Development of image preprocessing operations.
2. Implementation of the 23-feature extraction procedure.
3. Training and evaluation of SVM and Random Forest classifiers.
4. Integration of SHAP-based explanations.
5. Development of the Streamlit interface.
6. Integration of the modules into a complete verification pipeline.
7. Testing and correction of identified errors.
8. Preparation of the final project documentation.

Sl. No.	Activity	Expected Outcome
1	Image preprocessing	Processed signature images
2	Feature extraction	Numerical feature vectors
3	Model training and evaluation	Trained and evaluated classifiers
4	Explainability integration	SHAP-based explanations
5	Interface development	Streamlit web application
6	System integration	Complete verification workflow
7	Testing and evaluation	Identified and corrected issues
8	Documentation	Final project report

Table 5.2: Phase Two Project Activities

	Feb 3	Feb 9	Feb 23	Apr 8	May 12	Jun 10	July 7	Aug 13	Sept 7	Sept 18	Oct 3	Oct 7
Planning phase	☑	☑										
Literature Survey			☑									
Analysis phase				☑								
Design phase					☑							
Implementation						☑	☑	☑				
Testing									☑	☑		
Deployment											☑	
Documentation report												☑

Figure 5.4: Gantt Chart of Phase Two

5.3 Life Cycle Model

The life cycle model describes the stages followed during the development of ForgeXplain, from problem identification to the final testing and evaluation of the application.

An iterative development approach is suitable for the project because preprocessing, feature extraction, classification, and explainability may require repeated evaluation and improvement. Individual modules can be developed and tested separately before they are integrated into the complete system.

1. Requirement Analysis

The first stage identifies the problem of offline signature forgery detection and defines the objectives of the system. The functional requirements include image upload, preprocessing, feature extraction, classification, and explanation. Non-functional requirements include usability, reliability, maintainability, and security.

2. System Design

The architecture and interactions between the different modules are defined. The design specifies how signature images are processed, how features are supplied to the classifiers, and how predictions and explanations are displayed through the interface.

3. Data Preparation and Preprocessing

The CEDAR dataset is used for model development and evaluation. Signature images are prepared using resizing, grayscale conversion, denoising, adaptive thresholding, morphological operations, cropping, and recentering.

4. Feature Extraction and Model Development

A fixed set of 23 interpretable features is extracted from the processed images. The feature vectors are used to train and evaluate SVM and Random Forest classifiers.

5. Explainability and Interface Integration

The SHAP explanation module is integrated with the classification workflow. The Streamlit interface is developed to accept input images and display the predicted class and associated explanation.

6. Integration and Testing

The preprocessing, feature extraction, classification, explanation, and interface modules are combined into a single application. Testing is performed to identify issues in image handling, feature generation, model prediction, and result display.

7. Evaluation and Improvement

The performance of the classifiers is examined using the selected evaluation dataset. Identified problems are corrected where possible, and the system is refined to improve consistency, usability, and clarity.

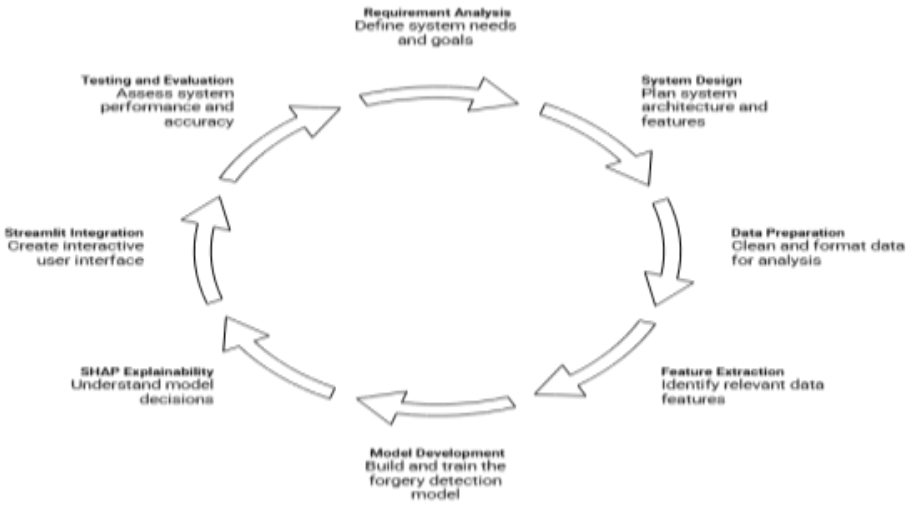


Figure 5.5: Life Cycle Model of ForgeXplain

5.4 Data Flow Diagram

A Data Flow Diagram (DFD) represents how data moves through the system and how the input is transformed into the final output. In ForgeXplain, the main input is a signature image, and the final output consists of the predicted signature class and its associated explanation.

The diagram includes the user, image preprocessing module, feature extraction module, machine learning classifier, SHAP explanation module, and result interface.

The main data flow is described below.

1. User Input: The user uploads a scanned signature image through the Streamlit interface.

2. Image Processing: The image is preprocessed to reduce noise, separate the signature from the background, and normalize its position.

3. Feature Extraction: The processed image is converted into a numerical representation containing 23 interpretable features.

4. Classification: The feature vector is supplied to the trained SVM or Random Forest classifier to obtain a prediction.

5. Explanation Generation: The SHAP module analyses the contribution of the input features to the model's prediction.

6. Result Presentation: The interface displays the predicted class and feature-level explanation to the user.

Data Element	Description
Signature Image	Image uploaded by the user
Preprocessed Image	Processed and normalized signature image
Feature Vector	Numerical representation containing 23 extracted features
Classification Output	Predicted genuine or forged class
SHAP Values	Feature contribution values associated with the prediction
Final Result	Prediction and explanation displayed through the interface

Table 5.3: Data Flow Description

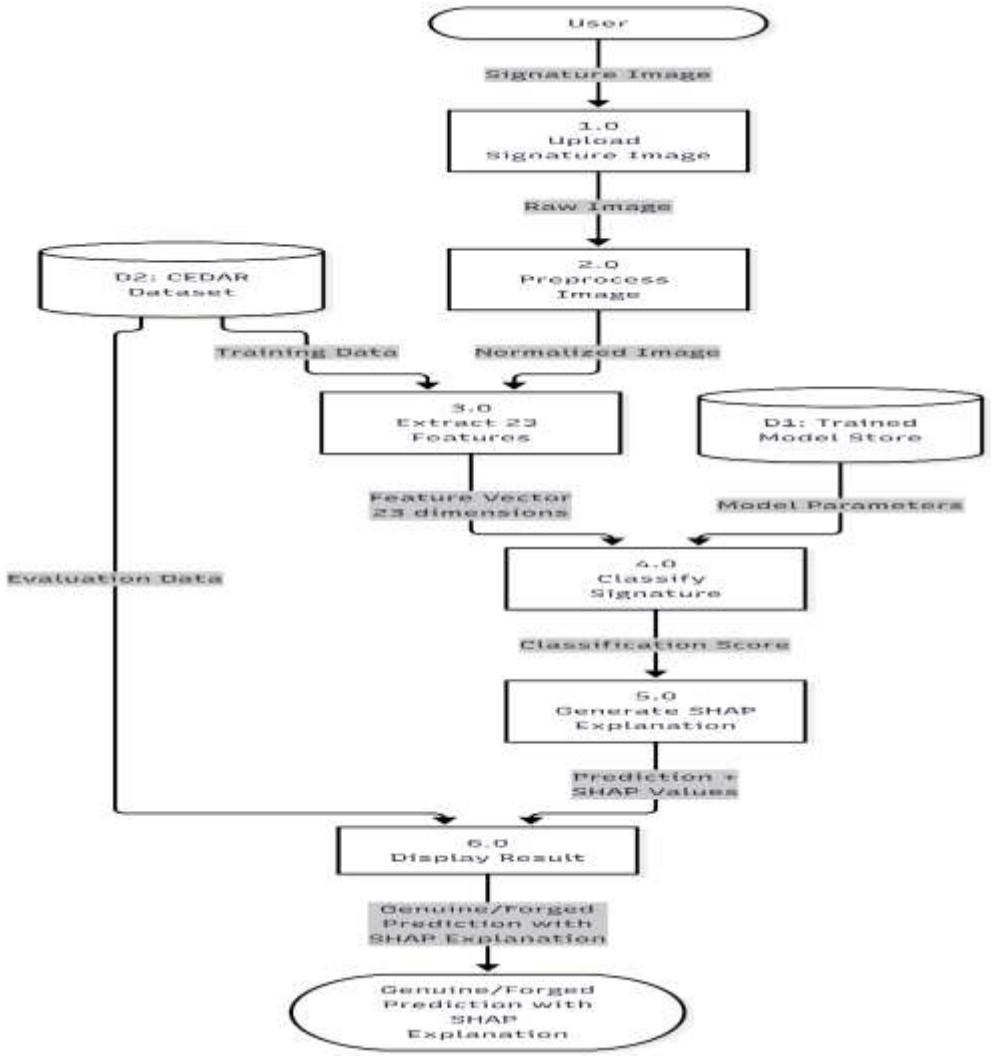


Figure 5.6: Data Flow Diagram of ForgeXplain

5.5 Use Case Diagram

A Use Case Diagram represents the interaction between the system and its users. It identifies the main activities that each actor can perform using ForgeXplain.

The two principal actors are the User and the Administrator.

1. User

The user interacts with the application to perform signature verification. The user can:

- Open the ForgeXplain application.
- Upload a signature image.
- Confirm or adjust a candidate signature region when required.
- Submit the image for verification.
- View the processed signature.
- View the predicted genuine or forged class.
- Examine available confidence information.
- View the SHAP-based feature explanation.

2. Administrator

The administrator performs restricted system-management activities according to the permissions configured in the application. These activities may include:

- Accessing administrative functions.
- Managing or inspecting trained models.
- Performing permitted model retraining.
- Accessing diagnostic information.
- Supporting system maintenance and troubleshooting.

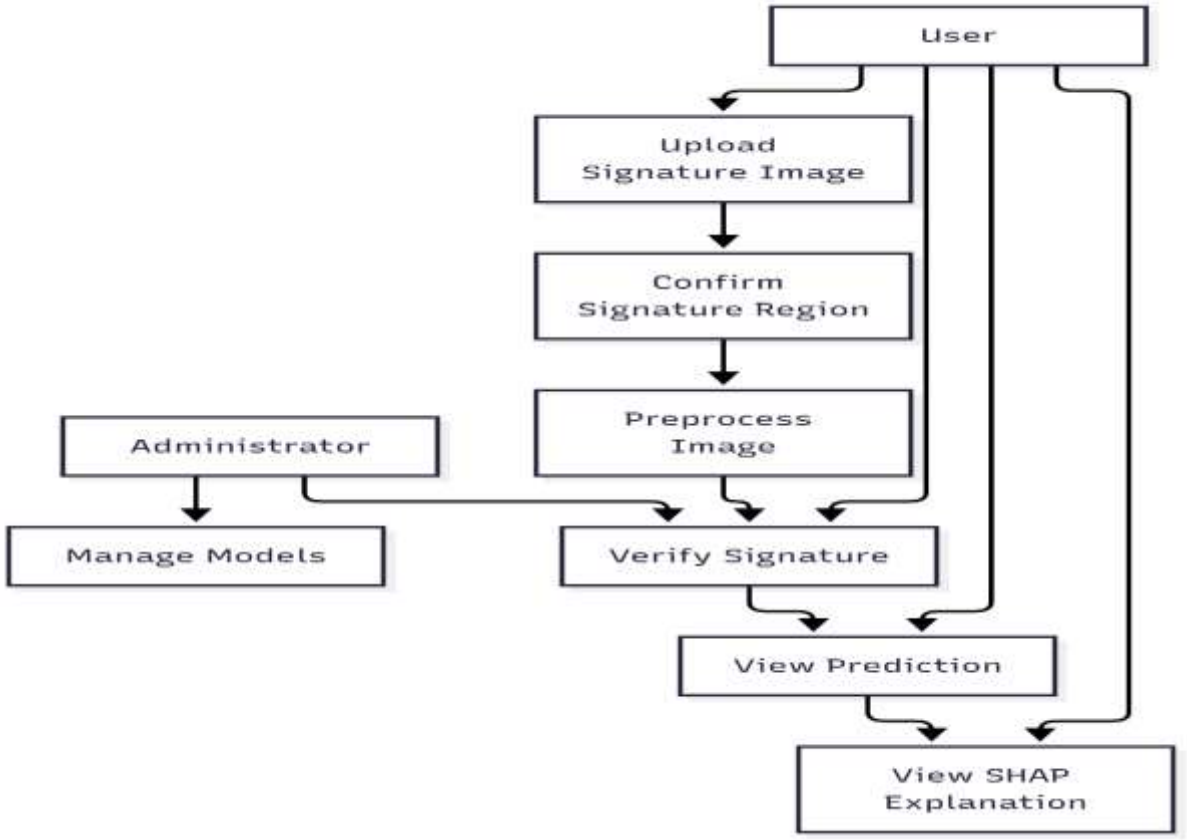


Figure 5.7: Use Case Diagram of ForgeXplain

5.6 Sequence Diagram

The sequence diagram illustrates the chronological interaction between the user and the major components of ForgeXplain during signature verification. It explains how the uploaded image passes through the system and how the prediction and explanation are returned to the user.

The main participants are the User, Streamlit Interface, Preprocessing Module, Feature Extraction Module, Machine Learning Classifier, SHAP Explanation Module, and Result Display.

Step 1: Upload Request

The user uploads a signature image through the Streamlit interface. The interface accepts the image and initiates the verification process.

Step 2: Image Preprocessing

The interface forwards the image to the preprocessing module. The module performs resizing, grayscale conversion, denoising, thresholding, morphological processing, and cropping or recentering as required.

Step 3: Feature Extraction

The processed image is passed to the feature extraction module. The module calculates the 23 features and returns the numerical feature vector.

Step 4: Classification Request

The feature vector is passed to the selected trained machine learning classifier. The classifier predicts whether the signature belongs to the genuine or forged class.

Step 5: Explanation Request

The relevant feature information and model prediction are used by the SHAP explanation module to generate feature contribution information.

Step 6: Result Preparation

The prediction, available confidence information, and explanation data are prepared for presentation.

Step 7: Result Display

The Streamlit interface presents the classification result and SHAP-based explanation to the user.

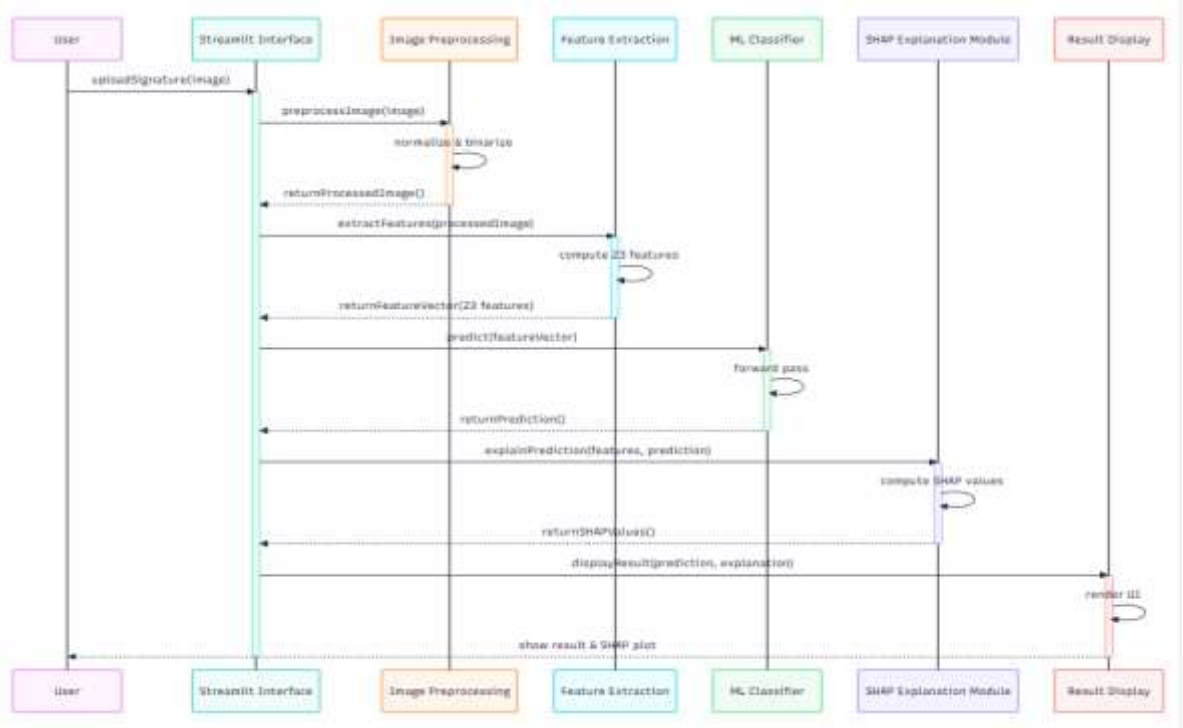


Figure 5.8: Sequence Diagram of ForgeXplain

5.7 Class Diagram

The class diagram represents the structural organization of ForgeXplain. It shows the main software components, their responsibilities, and their relationships. The system is organized around image preprocessing, feature extraction, classification, explanation generation, model management, and result presentation.

The following logical classes represent the main responsibilities of the application. They describe the system design and do not necessarily correspond to the exact names of classes in the source code.

1. UserInterface

The UserInterface component manages interaction between the user and the application.

Main responsibilities:

- Accept signature images.
- Initiate the verification process.
- Display the processed signature.
- Present classification results and explanations.
- Provide access to functions permitted for the user's role.

2. ImagePreprocessor

The ImagePreprocessor component prepares the uploaded signature image for feature extraction.

Main responsibilities:

- Resize the image.
- Convert the image to grayscale.
- Reduce noise using Gaussian blur.
- Apply adaptive thresholding.
- Perform morphological operations.
- Crop and recenter the signature.

3. FeatureExtractor

The FeatureExtractor component converts the processed image into a numerical feature representation.

Main responsibilities:

- Calculate shape and contour features.
- Calculate density and intensity-related features.
- Calculate histogram-based features and Hu moments.
- Return the 23-dimensional feature vector.

4. SignatureClassifier

The SignatureClassifier component performs the machine learning prediction.

Main responsibilities:

- Access the trained classifier.
- Receive the extracted feature vector.
- Predict the genuine or forged class.
- Return the prediction and supported confidence information.

5. ExplanationModule

The ExplanationModule component provides feature-level explanations using SHAP.

Main responsibilities:

- Obtain the information required to explain the prediction.
- Calculate or retrieve SHAP feature contributions.
- Identify features that influence the prediction.
- Return explanation data for display.

6. ModelManager

The ModelManager component is responsible for accessing and managing the trained models and related model operations.

Main responsibilities:

- Load or retrieve trained SVM and Random Forest models.
- Make the selected model available for prediction.
- Support permitted model-management functions.
- Assist with diagnostics and retraining where enabled.

7. ResultDisplay

The ResultDisplay component organizes the output for presentation through the interface.

Main responsibilities:

- Receive the classification result.
- Receive available confidence information.
- Receive the SHAP explanation.
- Present the result in an understandable format.

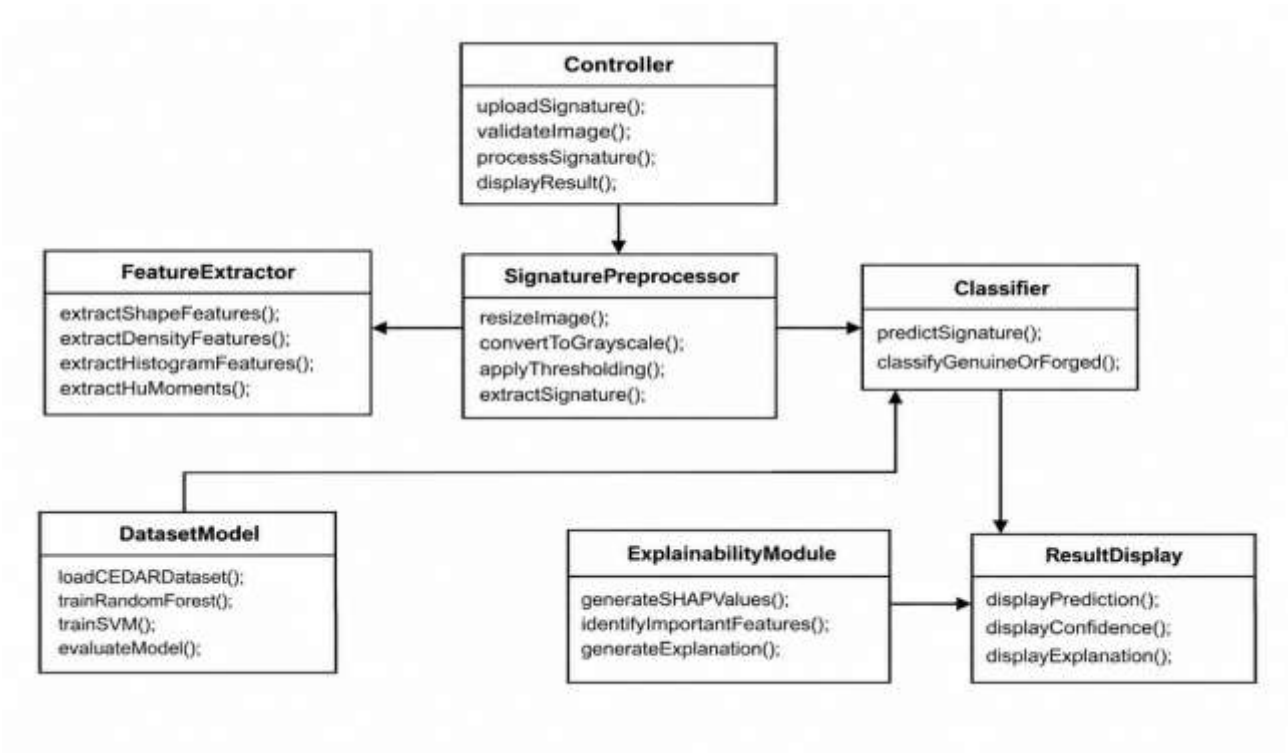


Figure 5.9: Class Diagram of ForgeXplain

Chapter 6

SYSTEM IMPLEMENTATION

The system implementation of ForgeXplain focuses on integrating image preprocessing, feature extraction, machine learning classification, and explainable artificial intelligence into a unified web-based application. The system is developed to detect possible forged signatures from scanned signature images and provide explanations for the predictions made by the machine learning model.

The application is implemented using Python and Streamlit. The machine learning component uses Support Vector Machine (SVM) and Random Forest classifiers, while SHAP is used to explain the contribution of features to the prediction. The implementation follows a modular approach in which each component performs a specific operation and passes its output to the next stage.

The complete workflow begins when the user uploads a signature image. The image is processed to improve its suitability for analysis, and 23 interpretable features are extracted. These features are supplied to the trained classifier, which predicts whether the signature is genuine or forged. Finally, the system presents the classification result and its associated explanation through the web interface.

6.1 Modular Approach

ForgeXplain follows a modular implementation approach in which the complete system is divided into smaller functional modules. Each module is responsible for a particular task, making the system easier to understand, maintain, test, and improve.

The major modules of the implemented system are the user interface module, image preprocessing module, feature extraction module, machine learning classification module, SHAP explanation module, and result display module. These modules work together to complete the signature verification process.

6.1.1 User Interface Module

The user interface is developed using Streamlit, which provides a web-based environment for interacting with the signature verification system. It allows the user to upload a scanned signature image and initiate the verification process.

The uploaded image is provided as input to the processing pipeline. The interface also presents the classification result and explanation generated by the system. This arrangement enables users to access the machine learning functionality without directly interacting with the underlying Python code.

When a signature is located inside a larger document, a candidate signature region can be identified for user confirmation or adjustment. This helps reduce the inclusion of unrelated document content during verification.

6.1.2 Image Preprocessing Module

The image preprocessing module prepares the uploaded signature image for feature extraction. Scanned images may contain noise, uneven backgrounds, unwanted borders, and differences in image size. These variations can affect the consistency of the extracted features.

The preprocessing pipeline includes image resizing, grayscale conversion, Gaussian blurring, adaptive Gaussian thresholding, morphological opening and closing, bounding-box cropping, and recentering, as applicable to the input image.

Resizing helps maintain a consistent image representation. Grayscale conversion simplifies the image by representing it through intensity values. Gaussian blurring helps reduce small amounts of noise, while adaptive thresholding separates the foreground signature from the background. Morphological operations help refine the binary representation, and cropping removes unnecessary empty regions around the signature.

The resulting image is passed to the feature extraction module. Appropriate preprocessing helps provide a more consistent input for the machine learning classifier.

6.1.3 Feature Extraction Module

The feature extraction module converts the preprocessed signature image into numerical data suitable for machine learning. ForgeXplain uses 23 interpretable features to represent the important characteristics of a signature.

The extracted features cover geometric properties, shape characteristics, pixel density, contour information, and intensity-histogram characteristics. Geometric features describe the overall dimensions and proportions of the signature. Shape features represent structural characteristics, while density and contour features provide information about the distribution and boundaries of the signature strokes. Histogram features represent the distribution of image intensities.

These features are combined into a numerical feature vector. The feature vector serves as the input to the trained classification model.

Using a compact feature representation also supports explainability because the model's decisions can be examined in terms of identifiable input features rather than only a final classification label.

6.1.4 Machine Learning Classification Module

The machine learning classification module determines the predicted category of the uploaded signature. ForgeXplain evaluates two supervised learning algorithms: Support Vector Machine and Random Forest.

Support Vector Machine (SVM): SVM learns a decision boundary that separates genuine and forged signatures using their extracted feature values. During verification, the trained model receives the feature vector of the uploaded image and produces a classification result.

Random Forest Classifier: Random Forest combines the predictions of multiple decision trees to determine the final classification. Its ability to model relationships between different feature values makes it suitable for evaluating the proposed signature representation.

Both models are trained and evaluated using the extracted features from the CEDAR signature dataset. The reported experimental results show that Random Forest performs better than SVM on the selected test set. Random Forest achieved 91.5% accuracy, whereas SVM achieved 86.4% accuracy.

Therefore, Random Forest is the better-performing model among the two evaluated classifiers under the reported experimental conditions.

6.1.5 SHAP Explanation Module

The SHAP explanation module provides information about the features that influence a machine learning prediction. A conventional classifier may return only the predicted category, making it difficult for users to understand the basis of the decision.

ForgeXplain integrates SHAP-based explanations to improve the interpretability of the classification process. SHAP estimates the contribution of individual features to a model's output. The explanation allows users to examine which features support or oppose a prediction under the model.

The project uses SHAP TreeExplainer for the Random Forest model. The feature-importance analysis identified hist_bin_2 as a major feature, accounting for 23.0% of the reported Random Forest feature importance. The first Hu moment accounted for 6.3%.

These values describe the reported model-level feature importance and are not the same as the SHAP contributions for an individual prediction. The explanation module provides additional information for reviewing the classification result, but it does not independently establish the authenticity of a signature.

6.1.6 Result Display Module

The result display module presents the output of the verification process through the Streamlit interface. After the image has been processed, the features extracted, and the classification completed, the predicted category is presented to the user along with the available explanation.

The result enables the user to review both the model's prediction and the features that influenced its decision. Displaying the explanation alongside the classification makes the system more transparent than an approach that presents only a genuine or forged label.

The result should be interpreted as a machine learning assessment. Errors may occur due to variations in handwriting, image quality, scanning conditions, or differences between the training dataset and real-world signatures.

6.2 Programming Code

```
<1DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
        <link rel="icon" type="image/svg+xml" href="data:image/svg+xml,<svg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 100 100'><text y='.9em' font-size='90'> ✍️ </text></svg>" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>ForgeXplain</title>
     <meta name="description" content="Explainable AI-Powered Offline Signature Forgery Detection System" />
    <meta property="og:title" content="ForgeXplain" />
     <meta property="og:description" content="Explainable AI-Powered Offline Signature Forgery Detection System" />
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <linkhref="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700;800&family=JetBrains+Mono:wght@400;500;600&display=swap" rel="stylesheet">
  </head>
  <body class="bg-[#0B0A14] text-[#EDEBF7] min-h-screen antialiased selection:bg-purple-600 selection:text-white">
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>


import React, { useState } from 'react';
import { AuthProvider, useAuth } from './context/AuthContext';
import { Sidebar } from './components/Sidebar';
```

```
import { HomeDashboard } from './pages/HomeDashboard';
import { SignatureDetection } from './pages/SignatureDetection';
import { Explainability } from './pages/Explainability';
import { ModelPerformance } from './pages/ModelPerformance';
import { PredictionHistory } from './pages/PredictionHistory';
import { AdminPanel } from './pages/AdminPanel';
import { UserProfile } from './pages/UserProfile';
import { AboutPage } from './pages/AboutPage';
import { AuthView } from './pages/AuthView';
import { AnalysisResult } from './types';
import { Menu } from 'lucide-react';
const DashboardContent: React.FC = () => {
  const { user, loading } = useAuth();
  const [currentTab, setCurrentTab] = useState<string>('home');
  const [isSidebarOpen, setIsSidebarOpen] = useState<boolean>(false);
  const [lastResult, setLastResult] = useState<AnalysisResult | null>(null);

  if (loading) {
    return (
      <div className="min-h-screen flex items-center justify-center bg-[#0B0A14] text-purple-300">
        <div className="w-8 h-8 border-3 border-purple-500/30 border-t-purple-500 rounded-full animate-spin" />
      </div>
    );
  }

  if (!user) {
    return <AuthView />;
```

```
  }
  return (
    <div className="flex min-h-screen bg-[#0B0A14] text-[#EDEBF7]">
      {/* Sidebar Navigation */}
      <Sidebar

     currentTab={currentTab}
       onSelectTab={setCurrentTab}
       isOpen={isSidebarOpen}
       onToggle={() => setIsSidebarOpen(!isSidebarOpen)}
      />

      {/* Main Content Area */}
      <div className="flex-1 flex flex-col min-w-0 overflow-y-auto">
        {/* Mobile Top Bar */}
        <header className="md:hidden flex items-center justify-between p-4 border-b border-purple-900/20 bg-[#100D1C]/80 backdrop-blur-md sticky top-0 z-30">
          <div className="flex items-center gap-2">
           <div className="w-8 h-8 rounded-lg bg-gradient-to-br from-violet-600 to-fuchsia-500 flex items-center justify-center text-white font-extrabold text-sm">
             Fx
           </div>
           <span className="font-extrabold text-sm text-purple-200">ForgeXplain</span>
          </div>
          <button
           onClick={() => setIsSidebarOpen(true)}
           className="p-2 rounded-lg bg-purple-900/30 text-purple-300 hover:text-white"
          >
           <Menu size={20} />
```

```
        </button>
      </header>

      {/* Content Container */}
      <main className="flex-1 p-4 sm:p-6 lg:p-8 max-w-7xl w-full mx-auto">
        {currentTab === 'home' && (
          <HomeDashboard onNavigate={(tab) => setCurrentTab(tab)} />
        )}

        {currentTab === 'detection' && (
          <SignatureDetection
            onAnalyzeComplete={(res) => setLastResult(res)}
          />
        )}

        {currentTab === 'explainability' && (
          <Explainability
            lastResult={lastResult}
            onNavigateToDetect={() => setCurrentTab('detection')}
          />
        )}
        {currentTab === 'performance' && <ModelPerformance />}

        {currentTab === 'history' && <PredictionHistory />}

        {currentTab === 'admin' && <AdminPanel />}

        {currentTab === 'profile' && <UserProfile />}
```

```
        {currentTab === 'about' && <AboutPage />}
      </main>
    </div>
  </div>
 );
};

export default function App() {
 return (

 <AuthProvider>
    <DashboardContent />
  </AuthProvider>
 );
}

"""
dataset_loader.py
--------------------
Loads signature images + genuine/forged labels for training.

Expected CEDAR layout (download separately - see dataset/README.md):
  dataset/CEDAR/
    full_org/    original_1_1.png ... original_55_24.png   (genuine)
    full_forg/   forgeries_1_1.png ... forgeries_55_24.png (forged)

BHSig260 / GPDS can be added later by writing one more `_load_<name>()`
```

```
method with the same return contract: (list_of_images, list_of_labels).
No other code needs to change — train_pipeline.py just calls load_dataset().

Since this sandbox has no access to download CEDAR, `synthetic` mode
generates a stroke-like signature dataset (via Bezier-curve renders +
controlled distortions for "forged" samples) so the whole pipeline —
feature extraction, training, evaluation, SHAP — runs end-to-end
immediately. Swap DATASET_SOURCE to "cedar" once the real files are in
dataset/CEDAR/.
"""
import os
import cv2
import numpy as np
from pathlib import Path
from typing import List, Tuple
from utils.config import CEDAR_DIR, SYNTHETIC_DIR, IMAGE_SIZE
from utils.logger import get_logger

logger = get_logger(__name__)

def _load_cedar() -> Tuple[List[np.ndarray], List[int]]:
    """label: 1 = genuine, 0 = forged"""
    images, labels = [], []
    genuine_dir = CEDAR_DIR / "full_org"
    forged_dir = CEDAR_DIR / "full_forg"

    for d, label in [(genuine_dir, 1), (forged_dir, 0)]:
```

```
    if not d.exists():
        continue
    for f in sorted(d.glob("*.png")) + sorted(d.glob("*.jpg")):
        img = cv2.imread(str(f), cv2.IMREAD_GRAYSCALE)
        if img is not None:
            images.append(img)
            labels.append(label)
    return images, labels
def _render_stroke_signature(rng: np.random.Generator, distort: bool = False) -> np.ndarray:
    """
    Renders a synthetic pen-stroke-like signature using random Bezier-ish
    polylines, optionally distorted (simulating a forged/imitated stroke)
}
```

6.3 System Implementation Summary

The implementation of ForgeXplain integrates image processing, feature-based machine learning, explainable AI, and a web-based user interface into a single signature verification workflow.The Streamlit interface provides a convenient method for submitting signature images and viewing the results. The preprocessing module prepares the image for analysis, and the feature extraction module generates 23 numerical features describing the signature's geometric, shape, density, contour, and intensity characteristics. The classification module evaluates the feature vector using trained machine learning models.The project evaluates SVM and Random Forest on the CEDAR dataset. The reported results show that Random Forest achieves an accuracy of 91.5%, precision of 91.3%, recall of 91.7%, F1-score of 91.5%, and ROC-AUC of 0.966. SVM achieves an accuracy of 86.4% and ROC-AUC of 0.939. These results indicate that Random Forest is the stronger of the two evaluated models for the reported experiment.The integration of SHAP adds an explanation layer that provides information about the contribution of features to model predictions. The application also supports user-assisted signature cropping and role-based access control, helping make the workflow more practical and easier to manage.Overall, the system implementation demonstrates how explainable smachine learning can be applied to offline signature forgery detection through a web-based application. Its performance should be interpreted in the context of the dataset and evaluation procedure used, and further validation on independent datasets is necessary before relying on the system in high-risk real-world settings.

CHAPTER 7

TESTING

7.1 Validation and System Testing

7.1.1 Software Testing

Software testing plays a crucial role in ensuring that the ForgeXplain system functions reliably, efficiently, and according to its design objectives. Since the proposed system consists of several interconnected modules such as image preprocessing, feature extraction, machine learning classification, explainability, and result visualization, testing was performed at different levels to verify the functionality, accuracy, and reliability of the complete system.

1. Objective of Software Testing

The main objective of software testing is to verify that all modules of the ForgeXplain system perform as expected and that their integration produces reliable signature verification results. Testing also helps identify errors in image processing, feature extraction, model prediction, and explanation generation.

The major objectives of software testing are:

- To verify that signature images are uploaded and processed correctly.
- To ensure that preprocessing produces suitable images for feature extraction.
- To verify that the required features are extracted from signature images.
- To evaluate the performance of SVM and Random Forest classifiers.
- To verify the generation of explainable predictions using SHAP.
- To ensure that the final prediction is displayed correctly.
- To identify and handle invalid inputs and processing errors.

2. Types of Testing Performed

To ensure comprehensive quality assurance, different types of testing were performed during the development and evaluation of ForgeXplain.

a) Unit Testing

Unit testing was performed on individual modules of the system. The dataset loading module was checked for correct loading and labeling of genuine and forged signature samples. The image preprocessing and feature extraction stages were checked to ensure that the input image was correctly transformed into the required numerical feature representation. The machine learning and explanation modules were also checked individually.

b) Integration Testing

Integration testing focused on verifying the data flow between the different modules. The uploaded signature image is passed through preprocessing, feature extraction, classification, and explanation stages. Integration testing ensured that the output generated by one module was correctly accepted as input by the next module without feature mismatch or processing errors.

c) System Testing

System testing was performed on the complete ForgeXplain application. The complete workflow, beginning with signature image upload and ending with prediction and explanation display, was checked to ensure that the system operated as an integrated application.

3. Testing Environment

Testing of ForgeXplain was performed using a computer environment capable of running the Python-based machine learning pipeline and the web-based application. The implementation uses Python-based machine learning libraries and a Streamlit interface for interaction with the system.

The system was evaluated using the CEDAR signature dataset containing genuine and forged signature samples. The dataset used in the reported evaluation consists of 2,640 images, including 1,320 genuine and 1,320 forged signatures. The data was divided into training and testing sets using an 80:20 stratified split.

The testing process included image uploading, preprocessing, feature extraction, classification using SVM and Random Forest, and generation of model explanations.

4. Testing Results

The following table summarizes the testing results obtained from the major modules of the ForgeXplain system.

Module	**Metric Evaluated**	**Result**
Image Upload Module	Input Image Acceptance	Successful
Image Preprocessing Module	Image Preprocessing	Successful
Feature Extraction Module	Number of Features Extracted	23 Features
SVM Classifier	Classification Accuracy	86.4%
SVM Classifier	ROC-AUC	0.939
Random Forest Classifier	Classification Accuracy	91.5%
Random Forest Classifier	Precision	91.3%
Random Forest Classifier	Recall	91.7%

Random Forest Classifier	F1-Score	91.5%
Random Forest Classifier	ROC-AUC	0.966
SHAP Explanation Module	Feature Explanation Generation	Successful
Result Display Module	Prediction Display	Successful

Table 7.1: Testing Results

The testing results indicate that the Random Forest classifier achieved better performance than the SVM classifier on the evaluated dataset. The Random Forest model achieved an accuracy of 91.5% and a ROC-AUC of 0.966, whereas the SVM model achieved an accuracy of 86.4% and a ROC-AUC of 0.939.

The Random Forest classifier also achieved a precision of 91.3%, recall of 91.7%, and F1-score of 91.5%. These results demonstrate the effectiveness of the implemented feature-based classification approach for the evaluated signature dataset.

5. Observations

The testing confirmed that the major modules of ForgeXplain operated together as a complete signature verification pipeline. The preprocessing stage prepared the uploaded signature images, while the feature extraction stage generated the required interpretable features for classification.

Among the evaluated models, Random Forest produced better reported classification performance than SVM. The explainability module further supports the system by providing information about the features contributing to the model prediction.

The testing also showed that the system can provide a complete workflow from signature image input to classification and explanation. However, the reported performance depends on the dataset and experimental setup used for evaluation.

7.1.2 Validation

Validation ensures that the developed ForgeXplain system performs according to its intended objectives and requirements. The validation process focuses on verifying whether the system can process offline signature images, classify them as genuine or forged, and provide meaningful information about the model's decision.

The validation of ForgeXplain was carried out by checking the complete processing pipeline, including image upload, preprocessing, feature extraction, machine learning classification, and explanation generation.

The classification results obtained from the evaluated dataset demonstrate that the system is capable of distinguishing between genuine and forged signatures. The Random Forest model provided the best reported performance, achieving 91.5% accuracy and a ROC-AUC of 0.966.

Validation also considered the explainability requirement of the system. SHAP-based feature explanations are generated to provide information about the contribution of extracted features to the prediction. This improves the interpretability of the machine learning-based signature verification process.

Overall, validation indicates that ForgeXplain meets its intended purpose of providing an offline signature forgery detection system with an explainable machine learning approach.

7.1.3 Reasons for Performing Software Validation

Software validation is performed to ensure that the ForgeXplain system is accurate, reliable, and aligned with its intended objectives. It verifies that the implemented modules work correctly and that the complete system produces the expected output.

The major reasons for performing software validation are:

- To verify that the system satisfies the specified functional requirements.
- To ensure that signature images are processed correctly.
- To verify that the extracted features are suitable for machine learning classification.
- To evaluate whether the trained models produce reliable predictions.
- To ensure that the explainability module provides meaningful feature information.
- To identify errors during integration of different system modules.
- To ensure that the final results are displayed correctly to the user.
- To improve confidence in the reliability and usability of the developed system.

Validation is particularly important for ForgeXplain because the system combines image processing, machine learning, and explainable artificial intelligence. Testing each stage and validating the complete workflow helps ensure that errors in one stage do not affect the final prediction.

7.2 Testing Summary

Testing summary provides an overview of the testing activities carried out for the ForgeXplain system. The different modules of the system were tested to ensure that they function correctly and work together as expected.

The testing process included verification of the signature image upload, image preprocessing, feature extraction, machine learning classification, explainability, and result display.

The complete workflow was also tested to ensure that the processed signature image is correctly passed through the feature extraction and classification stages.

The machine learning models were evaluated using standard performance measures. The SVM classifier achieved an accuracy of 86.4% with a ROC-AUC of 0.939, while the Random Forest classifier achieved an accuracy of 91.5% with a ROC-AUC of 0.966. The Random Forest model also achieved 91.3% precision, 91.7% recall, and 91.5% F1-score.

The SHAP-based explainability module was tested to ensure that important features influencing the model prediction could be identified and presented to the user. The final result display was also verified to provide the signature classification and corresponding explanation through the Streamlit interface.

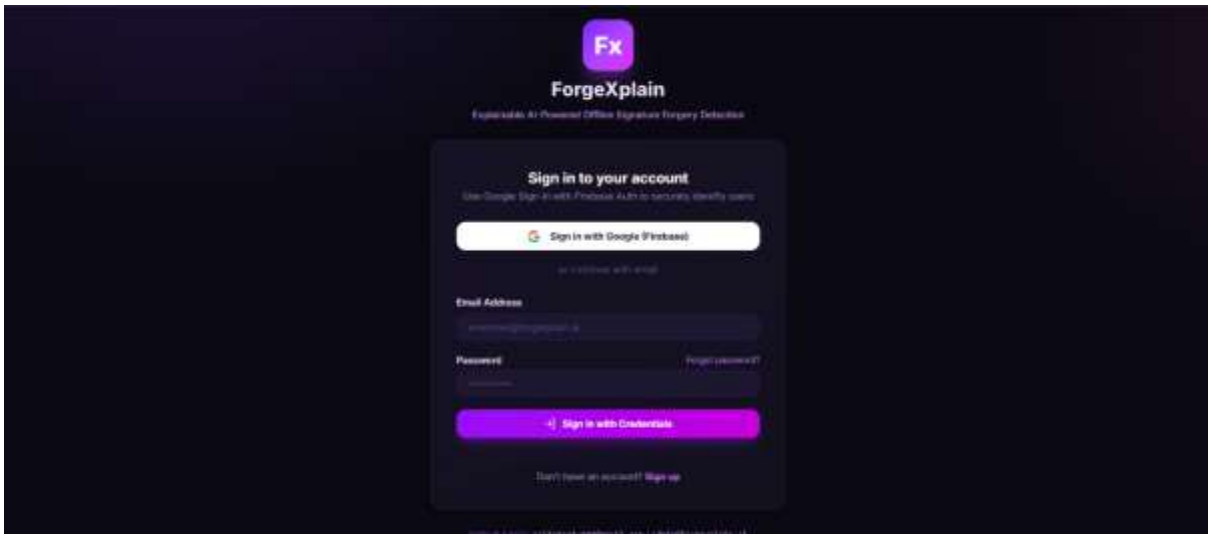
Overall, the testing process demonstrates that the major components of ForgeXplain are integrated into a functional workflow for offline signature forgery detection. The results also indicate that the Random Forest classifier provides better performance than the SVM classifier for the evaluated dataset.

CHAPTER 8

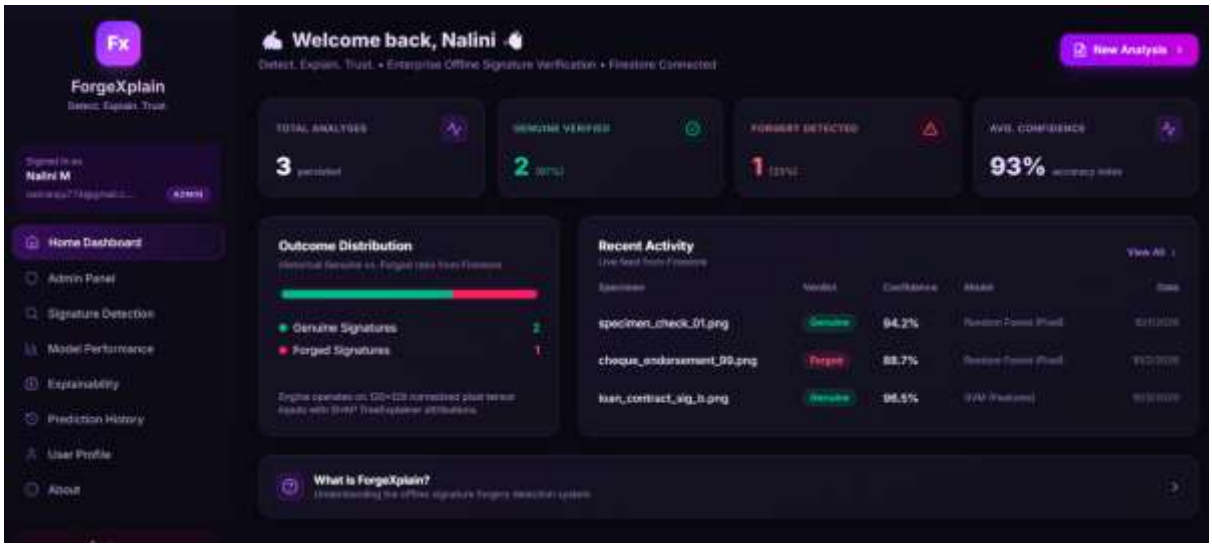
SAMPLE OUTPUT

8.1 Snapshots

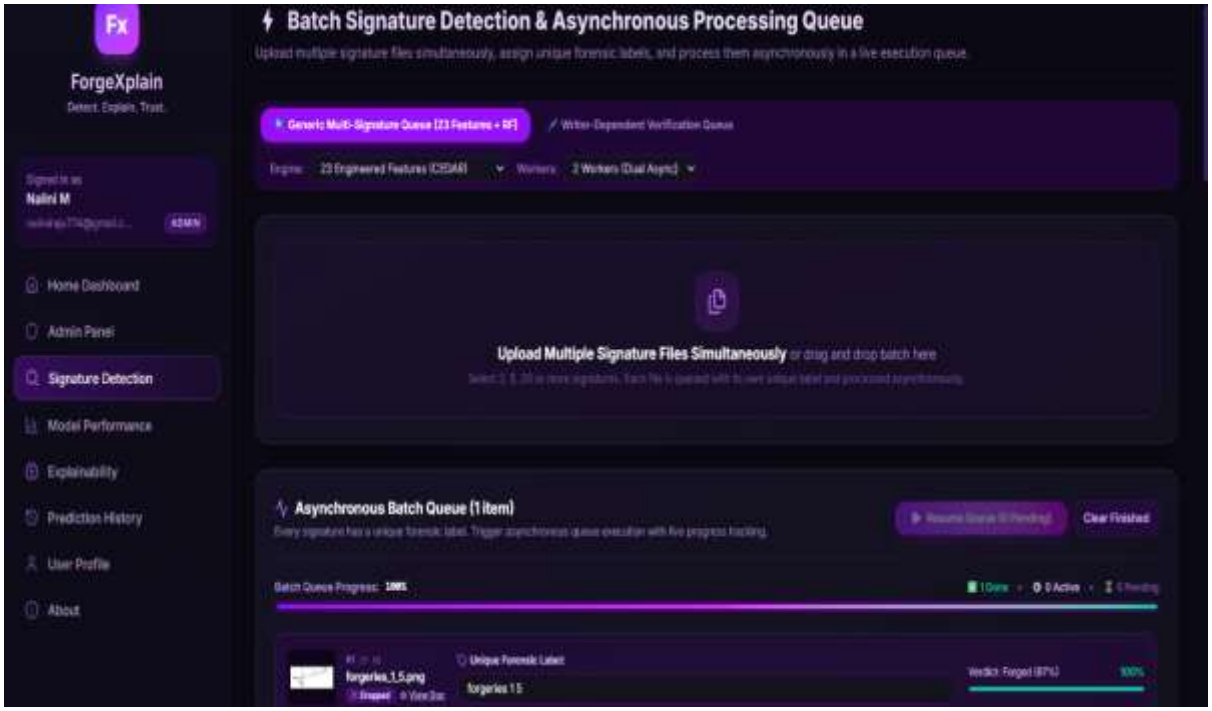
8.1.1 ForgeXplain Login Page



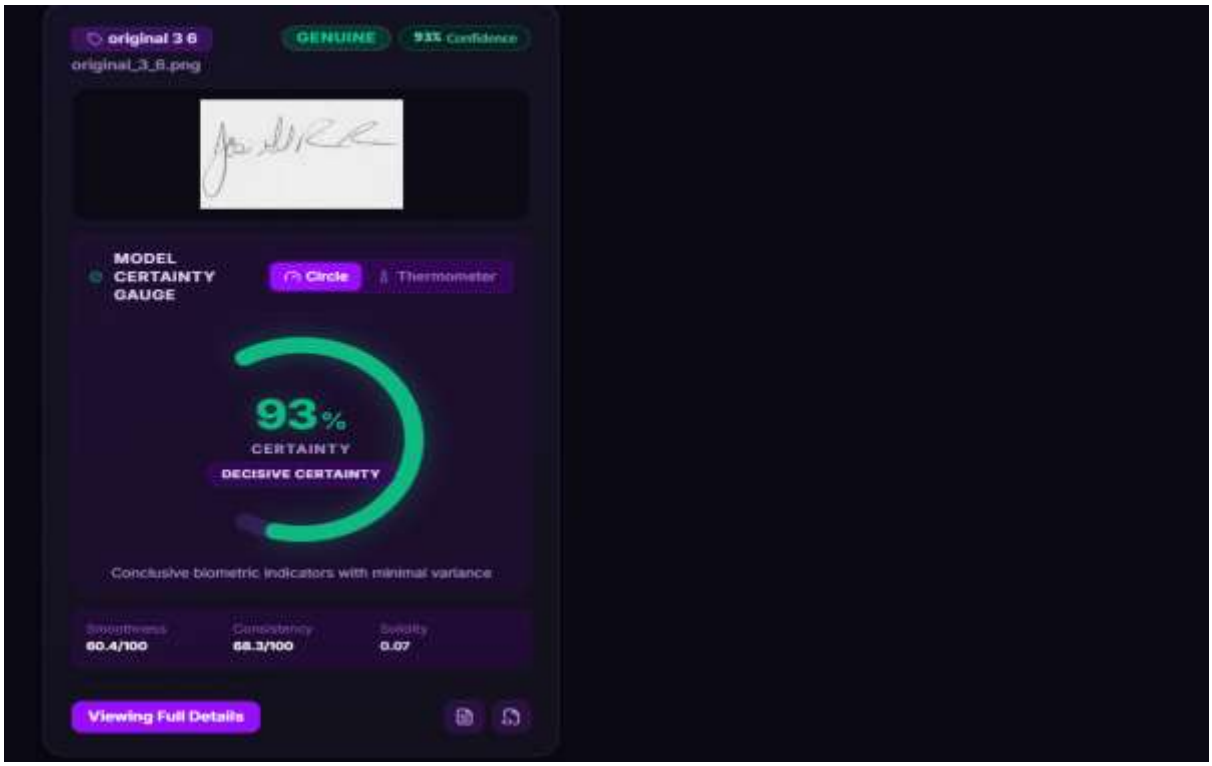
8.1.2 ForgeXplain Home Dashboard



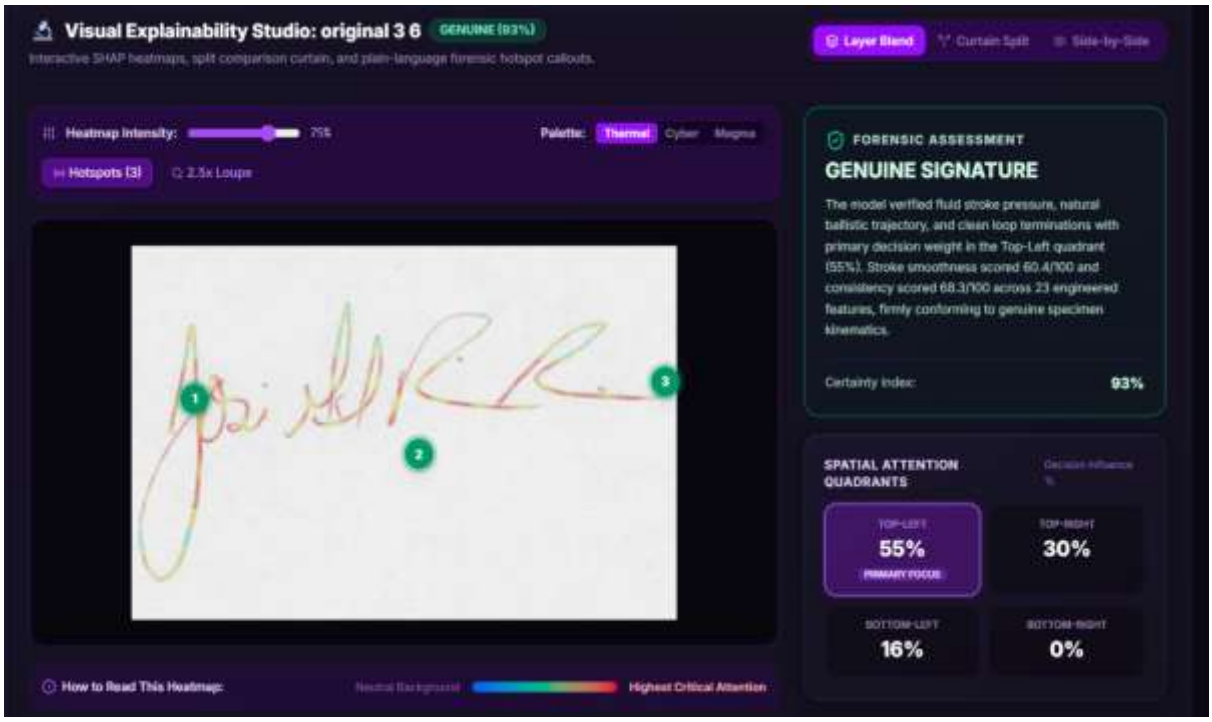
8.1.3 Signature Detection and Batch Processing Interface



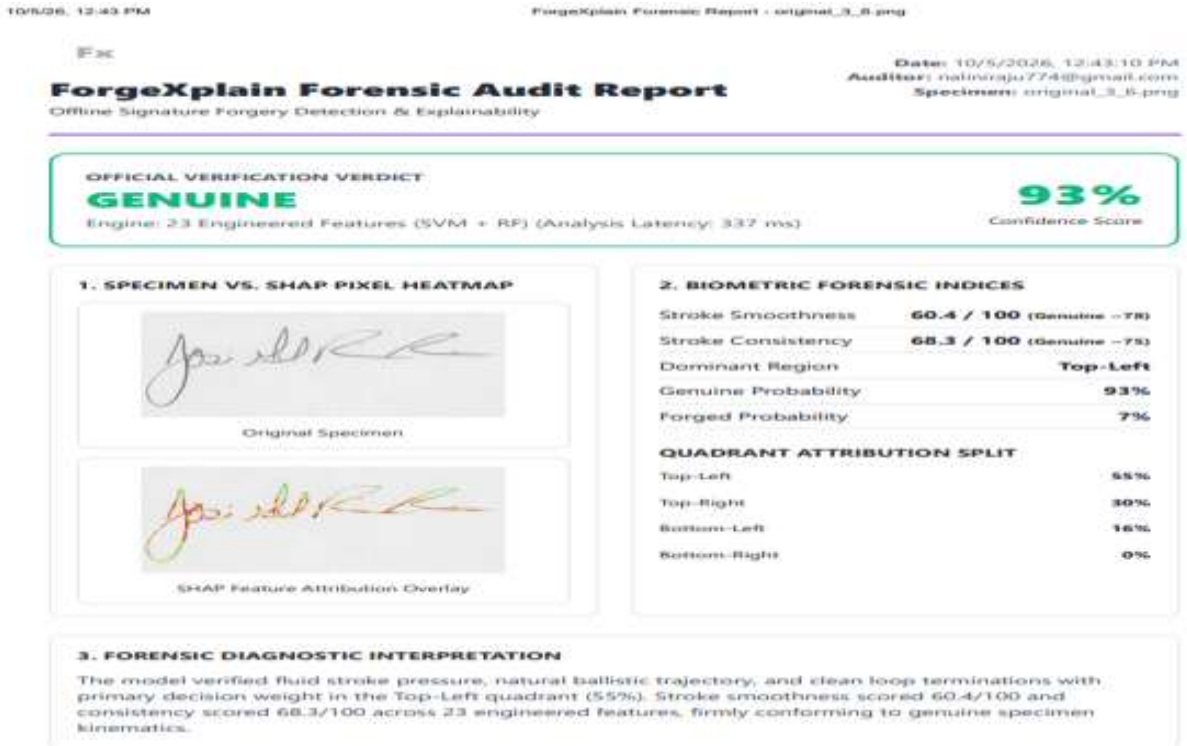
8.1.4 Genuine Signature Detection Result



8.1.5 SHAP-Based Visual Explainability of Signature Prediction



8.1.6 Generated ForgeXplain Forensic Audit Report



8.2 Sample Output Summary

The sample outputs demonstrate the working of the ForgeXplain system through its major functional components. The screenshots include the login page, home dashboard, signature detection and batch processing interface, signature classification result, SHAP-based visual explainability, and the generated forensic audit report.

The system successfully presents the uploaded signature, processes the image, predicts whether the signature is genuine or forged, and provides the corresponding confidence and explanation. The explainability module highlights the important features contributing to the prediction, while the forensic audit report provides a structured summary of the analysis.

Overall, the sample outputs demonstrate that ForgeXplain provides an integrated and user-friendly interface for offline signature forgery detection, prediction explanation, and forensic reporting.

CONCLUSION

ForgeXplain is an explainable machine learning framework developed for offline signature forgery detection and verification. The system integrates image preprocessing, extraction of 23 interpretable features, machine learning classification, SHAP-based explanation, and a Streamlit-based web interface. Among the evaluated models, Random Forest achieved the best performance with 91.5% accuracy and a ROC-AUC of 0.966, showing its effectiveness for the given signature dataset.The SHAP-based explanation module further improves the system by identifying the important features influencing each prediction. Thus, ForgeXplain provides not only signature classification but also a more transparent and understandable decision-making process.

Future Scope:

The system can be further improved by evaluating it on diverse datasets containing different writing styles, image qualities, and document conditions. Feature-selection techniques and alternative machine learning classifiers can be explored to improve classification performance.The explainability module can also be extended using additional explanation techniques. The preprocessing and cropping modules can be enhanced to handle documents containing multiple signatures, seals, and text regions. These improvements can make ForgeXplain more robust and suitable for practical signature verification and forensic applications.

REFERENCES

[1] Z. Wang, M. Muhammat, N. Yadikar, A. Aysa, and K. Ubul, "Advances in offline handwritten signature recognition research: A review," *IEEE Access*, vol. 11, pp. 120222–120236, 2023. DOI: 10.1109/ACCESS.2023.3326471.

[2] Z. Hashim, H. M. Ahmed, and A. H. Alkhayyat, "A comparative study among handwritten signature verification methods using machine learning techniques," *Sci. Program.*, vol. 2022, Art. no. 8170424, 2022. DOI: 10.1155/2022/8170424.

[3] P. Kiran, B. D. Parameshachari, J. Yashwanth, and K. N. Bharath, "Offline signature recognition using image processing techniques and back propagation neuron network system," *SN Comput. Sci.*, vol. 2, no. 3, Art. no. 196, 2021. DOI: 10.1007/s42979-021-00591-y.

[4] J. Poddar, V. Parikh, and S. K. Bharti, "Offline signature recognition and forgery detection using deep learning," *Procedia Comput. Sci.*, vol. 170, pp. 610–617, 2020. DOI: 10.1016/j.procs.2020.03.133.

[5] S. Dey, A. Dutta, J. I. Toledo, S. K. Ghosh, J. Lladós, and U. Pal, "SigNet: Convolutional Siamese network for writer independent offline signature verification," *Pattern Recognit. Lett.*, vol. 128, pp. 182–189, 2019. DOI: 10.1016/j.patrec.2017.07.024.

[6] T. M. Ghanim and A. M. Nabil, "Offline signature verification and forgery detection approach," in *Proc. 13th Int. Conf. Comput. Eng. Syst. (ICCES)*, 2018, pp. 293–298. DOI: 10.1109/ICCES.2018.8639420.

[7] M. Thenuwara and H. R. K. Nagahamulla, "Offline handwritten signature verification system using Random Forest classifier," in *Proc. 17th Int. Conf. Adv. ICT Emerg. Regions (ICTer)*, 2017, pp. 191–196. DOI: 10.1109/ICTER.2017.8257828.

[8] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," *Adv. Neural Inf. Process. Syst.*, vol. 30, pp. 4765–4774, 2017

[9] L. G. Hafemann, R. Sabourin, and L. S. Oliveira, "Learning features for offline handwritten signature verification using deep convolutional neural networks," *Pattern Recognit.*, vol. 70, pp. 163–176, 2017. DOI: 10.1016/j.patcog.2017.05.012.

[10] A. Guerbai, Y. Chibani, and B. Hadjadji, "The effective use of One-Class SVM classifier for handwritten signature verification based on writer-independent parameters," *Pattern Recognit.*, vol. 48, no. 1, pp. 103–113, 2015. DOI: 10.1016/j.patcog.2014.07.015.

[11] R. Wajid, H. Malik, and M. I. Malik, "Offline signature verification using local binary pattern features," *Int. J. Pattern Recognit. Artif. Intell.*, vol. 28, no. 7, Art. no. 1456003, 2014. DOI: 10.1142/S0218001414560036.

[12] S. Y. Ooi, A. B. J. Teoh, and Y. H. Pang, "Radon transform based approach for offline signature verification," *Expert Syst. Appl.*, vol. 40, no. 17, pp. 6838–6846, 2013. DOI: 10.1016/j.eswa.2013.06.042.

[13] O. O. Khalifa, A. H. A. Hashim, and S. A. Samad, "Offline signature verification and forgery detection: A review," *International Journal of Computer Science and Network Security*, vol. 11, no. 3, pp. 170–176, 2011.

[14] R. L. Larkins and M. Mayo, "Adaptive feature thresholding for off-line signature verification," in *Proc. 23rd Int. Conf. Image Vis. Comput. New Zealand (IVCNZ)*, 2008, pp. 1–6. DOI: 10.1109/IVCNZ.2008.4762072.

ARTIFICIAL INTELLIGENCE, MACHINE LEARNING

ForgeXplain: An Explainable Machine Learning System for Offline Signature Forgery Detection

Madiha Kaunain[1], M. Nalini[2], Noor Ayesha[2], R. Saaranya[2]

[1]Department of Computer Science and Engineering, Rajiv Gandhi Institute of Technology, Bengaluru, India
[2]Rajiv Gandhi Institute of Technology, Bengaluru, India

Published October 2026

Abstract

Achieving accurate signature verification is critical in many contexts including financial transactions and legal contracts, for reasons of security and fraud risk. The authors describe an offline signature verification system called ForgeXplain that uses hand engineered image features in conjunction with machine learning algorithms and explainable AI. From pre-processed signature images the system extracts 23 hand designed geometric and intensity features that in turn feed into support vector machine (SVM) and random forest (RF) models in order to predict fake from real signatures, with SHAP values explaining the decision on a feature-by-feature basis. A test of 1,320 real and 1,320 fake signatures from CEDAR offline found that Random Forest was correct 91.5 percent of the time and achieved a ROC-AUC score of 0.966, better than SVM. This shows that ForgeXplain can tackle off-line signature forgery detection effectively and faithfully.

Keywords: signature verification; forgery detection; explainable AI; SHAP; random forest; support vector machine; CEDAR dataset

1 Introduction

Handwritten signatures are commonly used to verify identity in banking, legal agreements, academic records, and other sensitive transactions. Since signatures can be forged, fraudulent use may result in financial loss, impersonation, or legal disputes. Traditionally, verification is performed by comparing a questioned signature with known genuine samples. However, this manual process is time-consuming and difficult to automate because handwriting varies between individuals and examiner judgements may be subjective [1, 2, 13, 14].

Signature verification is divided into online and offline approaches. Online methods use dynamic information recorded during the signing process, whereas offline methods examine a static signature image obtained through scanning or photography. Therefore, offline verification relies heavily on the visual features extracted from the image, making feature selection an important factor in system performance [1, 2, 11, 13, 14].

Several studies have applied handcrafted geometric, structural, texture, and directional features with traditional machine-learning classifiers, including Random Forest and Support Vector Machine, to distinguish genuine signatures from forged ones [3–6, 9, 11, 13]. Deep-learning approaches have also been investigated for automatically learning discriminative representations from signature images [7, 8, 12, 14]. However, many automated systems provide only a classification result without explaining the features responsible for the prediction. This limitation may reduce user confidence and make it more difficult to assess incorrect decisions.

To address this issue, ForgeXplain is proposed as an explainable offline signature verification system. The framework combines image preprocessing, handcrafted feature extraction, machine-learning classification, and explainable artificial intelligence. It extracts 23 features related to geometry, shape, density, contour, and intensity, and evaluates Support Vector Machine (SVM) and Random Forest (RF) classifiers for distinguishing genuine and forged signatures. SHAP-based explanations are used to identify the contribution of individual features to each classification decision.

The proposed system is evaluated using the CEDAR offline signature dataset, which contains 1,320 genuine and 1,320 forged signature samples. The evaluation considers both classification performance and the interpretability of the model predictions.

2 Related work

Research into offline handwritten signature verification has explored the use of image processing, handcrafted features, conventional machine learning, and deep learning. Khalifa and colleagues [1] provide a comprehensive review of major offline signature verification and forgery detection techniques, encompassing preprocessing, feature extraction, and classification. Ghanim and Nabil [2] demonstrated the application of image-processing and machine-learning techniques for distinguishing between genuine and forged signatures, where Thenuwara and Nagahamulla [3] highlighted the effectiveness of Random Forest when paired with appropriate signature features. Larkins and Mayo [4] introduced adaptive feature thresholding to enhance the discrimination between genuine and forged items. Approaches based on texture and shape have also been investigated. Wajid and colleagues [5] investigated Local Binary Pattern features for capturing local texture information, whereas Ooi et al. [6] employed the Radon transformation to isolate directional features. With the advancement of deep learning, Hafemann et al. [7] demonstrated the ability of convolutional neural networks to learn discriminative signature features, and Dey et al. [8] introduced SigNet, a convolutional Siamese network designed for writer-independent verification. Guerbai et al. [9] further investigated the application of one-class SVM for writer-independent verification. To enhance model interpretability, Lundberg and Lee [10] introduced SHAP, a method that assigns contribution values to input features, thereby enabling explanations of machine-learning predictions at feature level. Other studies such as those by Kiran et al. [11], Poddar et al. [12], Hashim et al. [13], Wang et al. [14] further investigated image processing, neural networks, machine-learning classifiers and the shift towards deep-learning-based signature verification.

Research gap: While existing studies have demonstrated the promising verification performance, they primarily focus on classification accuracy and offer limited insight into the reasoning behind individual predictions. While SHAP facilitates interpretable machine-learning decisions [10], its application in offline signature forgery detection is still limited. This drives the development of ForgeXplain, a method that integrates signature forgery detection with feature-level explanations to deliver predictions that are both accurate and interpretable.

3 System architecture and methodology

ForgeXplain is built as a multi-page Streamlit web application that leverages a scikit-learn-based pipeline for both training and inference. The overall workflow is depicted in Figure 1.

The system handles an uploaded signature by sequentially executing preprocessing, feature extraction, classification, and explainability stages. The final output includes the predicted class, confidence score, and feature-level contributions, all displayed via the application interface.

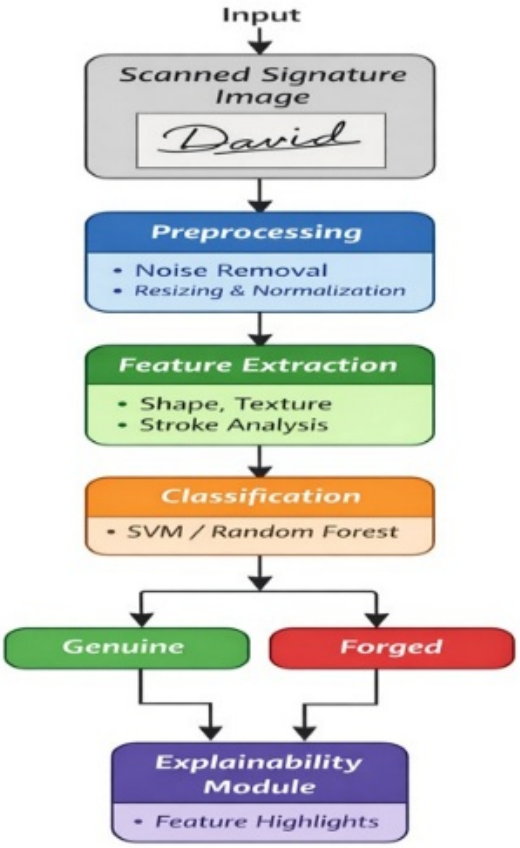


Figure 1. ForgeXplain system architecture for signature forgery detection.

3.1 Preprocessing and segmentation

The signatures are resized, converted to grayscale, and denoised using Gaussian blur. The image is first binarized using adaptive thresholding, and then morphological operations are applied to remove noise while preserving the strokes. The signature is cropped and re-centered to ensure consistency. For documents, the system automatically detects a candidate region, which the user then confirms to filter out irrelevant content.

3.2 Feature extraction

ForgeXplain extracts a fixed 23-dimensional feature vector from each preprocessed signature. These features capture complementary geometric, shape, density, contour, and intensity characteristics while maintaining interpretability for subsequent explanations. The features are grouped into five categories:

- **Geometric features (5):** contour area, perimeter, aspect ratio, bounding-box width, and height capture the signature's size and geometric proportions.
- **Shape features (7):** seven log-scaled Hu moment invariants capture the overall signature shape while providing invariance to translation, scaling, and rotation.

- **Density features (2):** the foreground pixel density and the black-to-white pixel ratio serve as indicators of ink coverage within the signature area.
- **Contour features (3):** the count of unique contours, mean contour area, and solidity reflect continuity, fragmentation, and structural properties.
- **Intensity histogram features (6):** a six-bin normalized grayscale histogram captures the distribution of pixel intensities.

This feature set strikes a balance between discriminative power and interpretability. In contrast to high-dimensional representations, which can be challenging to connect with visible signature characteristics, these features possess direct geometric or visual interpretations that can be seamlessly integrated into the explainability layer.

3.3 Classification

The extracted feature vectors are standardized using a scaler fitted solely on the training data and subsequently fed into two independently trained classifiers: a Support Vector Machine (SVM) and a Random Forest (RF).

The SVM employs an RBF kernel with a C value of 2.0 and has probability estimation enabled. The Random Forest model comprises 300 decision trees. Since both classifiers employ the identical feature representation and train-test split, their performance can be compared consistently.

3.4 Explainability layer

ForgeXplain incorporates SHAP-based explainability directly into the prediction pipeline. For the Random Forest classifier, TreeSHAP leverages the structure of the decision-tree ensemble to efficiently compute feature-level Shapley values.

For every analyzed signature, the explainability layer pinpoints the features driving the predicted class and quantifies both the direction and magnitude of their impact on distinguishing between genuine and forged signatures. These explanations are displayed within the application interface alongside the classification results, thereby embedding interpretability directly into the prediction workflow.

The result interface also presents the processed signature image, enabling users to review the analyzed image alongside its corresponding feature-level explanation.

4 Experimental results and analysis

4.1 Dataset

ForgeXplain is evaluated on the CEDAR offline signature dataset, which comprises 2,640 signature images in total—1,320 genuine and 1,320 forged signatures collected from 55 writers. The dataset includes 24 genuine and 24 forged samples for each writer. The dataset is employed to assess the performance of the proposed signature verification system as well as the two selected classification models.

4.2 Experimental setup

The dataset is partitioned into training and testing sets via an 80:20 stratified random split performed at the image level. This yields 2,112 training images and 528 test images, each subset

preserving an equal balance of genuine and forged signatures. Both the SVM and Random Forest classifiers were trained using the identical 23-dimensional feature representation detailed in Section III-B, along with the configurations specified in Section III-C, and all performance metrics were evaluated on the held-out test set comprising 528 images.

4.3 Classification results

Table 1 summarizes the classification performance of the SVM and Random Forest models, reporting metrics for accuracy, precision, recall, F1-score, and ROC-AUC.

Table 1. Classification performance on the CEDAR test set.

Classifier	Accuracy	Precision	Recall	F1-score	ROC-AUC
SVM (RBF)	86.4%	86.1%	86.7%	86.4%	0.939
Random Forest (300 trees)	91.5%	91.3%	91.7%	91.5%	0.966

The SVM misclassified 37 forged signatures as genuine and 35 genuine signatures as forged. These results suggest that both classifiers achieved relatively balanced performance, although the Random Forest model provided better overall accuracy for the selected feature representation.

4.4 Feature importance and explainability

The Random Forest analysis showed that intensity-histogram features and the first Hu moment had the greatest influence on classification. As shown in Table 2, hist_bin_2 was the most important feature, contributing 23.0% to the model's decisions. The first Hu moment was the most influential non-histogram feature, with an importance of 6.3%.

Table 2. Top six Random Forest feature importances.

Rank	Feature	Relative importance
1	hist_bin_2 (ink-density histogram)	23.0%
2	hist_bin_3 (ink-density histogram)	9.2%
3	hist_bin_5 (ink-density histogram)	6.8%
4	hist_bin_4 (ink-density histogram)	6.7%
5	hu_moment_1 (overall shape)	6.3%
6	hist_bin_6 (ink-density histogram)	5.4%

These results demonstrate the value of ForgeXplain's explainability module. In addition to predicting whether a signature is genuine or forged, the system identifies the features that influence each decision, thereby supporting more transparent verification and user review.

These findings demonstrate the usefulness of the explainability component in identifying the characteristics that influence the classifier. Rather than providing only the final classification, ForgeXplain allows the contribution of individual features to be examined for a given prediction. This provides additional information for users reviewing the output of the signature verification system. An example of the local SHAP explanation is shown in Figure 2.

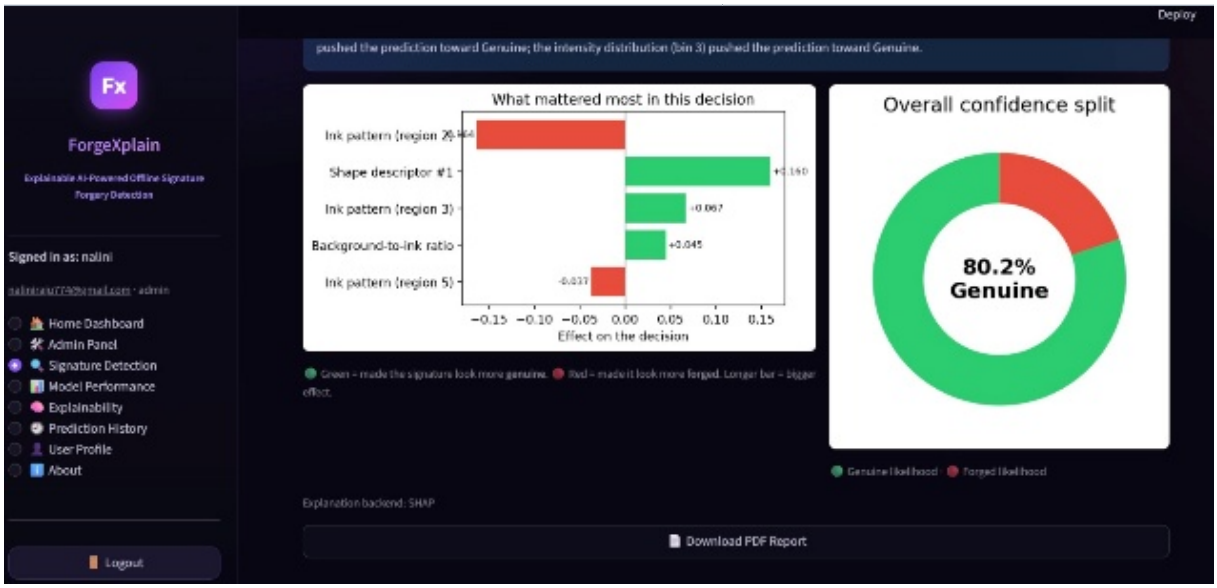


Figure 2. Local SHAP explanation interface illustrating individual feature impacts (ink patterns and region-based shape descriptors) on model decision outputs.

4.5 Comparison with prior work

Table 3 compares ForgeXplain's accuracy with that of selected offline signature-verification systems. This comparison is intended only to provide context, since the studies used different datasets, feature representations, training methods, and evaluation protocols. Consequently, the reported accuracy values should not be considered directly equivalent.

Table 3. Comparison with reported offline signature verification results.

System	Dataset	Features	Best accuracy
Ghanim and Nabil [2]	UTSig	HOG + SVM	94.0%
Larkins and Mayo [4]	In-house dataset (2,106 signatures)	Modified direction feature + RBF-NN	91.2%
Thenuwara and Nagahamulla [3]	ICDAR	Random Forest	67.0%
ForgeXplain (this work)	CEDAR	Geometric/shape/density/histogram + Random Forest	91.5%

Ghanim and Nabil [2] evaluated an offline signature verification system using the UTSig dataset and various feature–classifier combinations. Their highest accuracy was 94%, achieved using an SVM with Histogram of Gradients features.

Larkins and Mayo [4] introduced Adaptive Feature Thresholding (AFT) and tested it on the CEDAR and GPDS datasets. Their CEDAR results were 92% for the manual version and 90% for the automatic version; therefore, the 92% result is included in Table 3 for comparison.

Thenuwara and Nagahamulla [3] also assessed several classifiers, with their Random Forest model achieving 60% accuracy.

ForgeXplain's Random Forest model achieved 91.5% accuracy on the CEDAR test set, which is comparable to earlier results. However, these values should be interpreted cautiously because the studies

use different datasets, features, training methods, and evaluation procedures. The CEDAR-based result reported by Larkins and Mayo offers the most relevant comparison. ForgeXplain focuses not only on classification accuracy but also on interpretable features and SHAP-based explanations, which link feature characteristics to individual predictions. Thus, its main contribution is the integration of signature verification with interpretability rather than accuracy optimization alone.

5 Practical deployment considerations

ForgeXplain is designed for practical use as a client-facing web application, with emphasis on response time, reliable preprocessing, and secure system management.

5.1 Inference latency

The trained models and SHAP Tree Explainer are loaded once and retained in memory for subsequent requests. This avoids repeated disk access and reduces processing overhead. Under the tested environment, post-initialization model retrieval and inference were completed within the sub-millisecond range, although the first request incurred additional cold-start time.

5.2 Human-assisted signature cropping

For documents containing signatures within larger regions, the system detects a candidate signature area for user confirmation or adjustment. This step helps exclude unrelated elements such as text, stamps, and borders, improving the quality of the image supplied to the feature-extraction stage (Figure 3).

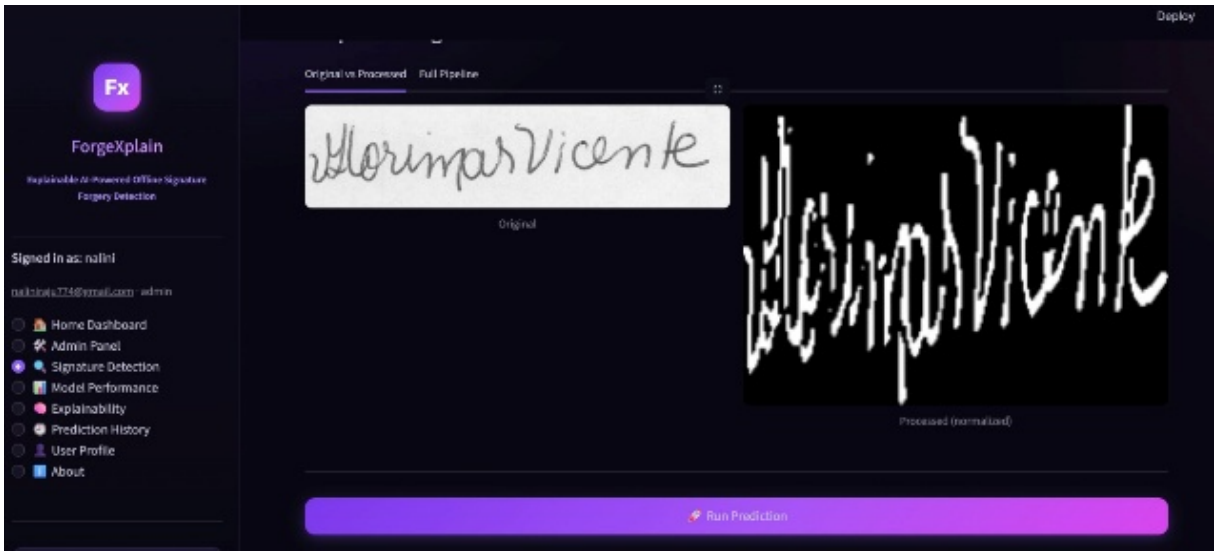


Figure 3. Interactive Streamlit application illustrating user-assisted signature upload, candidate region cropping, and prediction confidence scoring.

5.3 Access control

ForgeXplain uses role-based access control to separate ordinary verification tasks from administrative functions. General users can verify signatures, while administrators can access restricted features such as retraining, diagnostics, and model management. This separation reduces the risk of unauthorized changes to the models and system configuration. A generated analysis report is shown in Figure 4.



ForgeXplain — Signature Analysis Report

Generated for: naliniraju774@gmail.com
Timestamp: 2026-08-22 12:25:49

Uploaded Signature

Prediction Summary

Field	Value
Prediction	Genuine
Confidence	69.8%
Model Used	Random Forest
Prediction Time	146.75 ms

Explainable AI Summary

This prediction was primarily driven by: the intensity distribution (bin 2) pushed the prediction toward Genuine; the intensity distribution (bin 3) pushed the prediction toward Genuine; the shape descriptor #1 (Hu moment) pushed the prediction toward Forged.

Feature	Contribution
Hist Bin 2	+0.1795
Hist Bin 3	+0.0832
Hu Moment 1	-0.0449
Hu Moment 3	-0.0374
Hist Bin 5	-0.0366

This report was generated automatically by ForgeXplain's explainable AI pipeline. Results should be used as a decision-support tool alongside expert forensic review.

Figure 4. Generated ForgeXplain signature analysis report detailing verification outcome, model metrics, and feature-level explainable AI (XAI) contributions.

6 Discussion

6.1 Interpretation of results

Random Forest performed better than SVM with the proposed 23-feature representation. It achieved 91.5% accuracy, 91.3% precision, 91.7% recall, 91.5% F1-score, and a ROC-AUC of 0.966, compared with 86.4% accuracy and a ROC-AUC of 0.939 for SVM. The balanced precision and recall indicate reliable identification of both genuine and forged signatures, confirming the usefulness of the extracted geometric, shape, density, contour, and intensity features on the CEDAR dataset.

6.2 Accuracy-interpretability trade-off

ForgeXplain combines competitive classification performance with interpretable features. Instead of relying on complex, high-dimensional representations, it uses 23 meaningful features that can be directly examined by the explanation module. The Random Forest classifier achieved 91.5% accuracy while retaining greater interpretability than typical black-box approaches.

6.3 SHAP-based explainability

SHAP was used to explain individual predictions by measuring the positive or negative contribution of each feature. The results identified hist_bin_2, other intensity-histogram features, and the first Hu moment as major contributors to the model's decisions. These explanations support the review of

automated verification results. However, they remain dependent on the trained model and evaluation conditions.

6.4 Practical implications and limitations

The Streamlit implementation integrates signature classification, user-assisted cropping, SHAP explanations, and role-based access control into a single workflow. Nevertheless, the evaluation used only the CEDAR dataset and an image-level 80:20 train–test split. Therefore, the results may not fully represent performance under different writers, documents, or acquisition conditions. Validation using multiple datasets and writer- and document-independent protocols is required.

7 Conclusion and future work

ForgeXplain is an explainable framework for offline signature verification that combines feature extraction, machine learning, SHAP-based explanations, and web-based deployment. Using 23 extracted features, the study evaluated SVM and Random Forest classifiers on the CEDAR dataset. Random Forest achieved the best results, with 91.5% accuracy, 91.3% precision, 91.7% recall, 91.5% F1-score, and a ROC-AUC of 0.966. The Streamlit application further integrates signature cropping, instance-level explanations, verification, and role-based access control into a unified system.

Future work will evaluate the framework on diverse datasets covering different writing styles, image qualities, and document conditions. Further improvements may include feature-selection methods, alternative classifiers, model-agnostic explanation techniques, and enhanced cropping for pages containing multiple signatures, seals, and text regions.

References

[1] Khalifa OO, Hashim AHA, Samad SA. Offline signature verification and forgery detection: A review. *International Journal of Computer Science and Network Security*. 2011;11(3):170–176.

[2] Ghanim TM, Nabil AM. Offline signature verification and forgery detection approach. In: *2018 13th International Conference on Computer Engineering and Systems (ICCES)*; 2018. p. 293–298. https://doi.org/10.1109/ICCES.2018.8639420

[3] Thenuwara M, Nagahamulla HRK. Offline handwritten signature verification system using Random Forest classifier. In: *17th International Conference on Advances in ICT for Emerging Regions (ICTer)*; 2017. p. 191–196. https://doi.org/10.1109/ICTER.2017.8257828

[4] Larkins RL, Mayo M. Adaptive feature thresholding for off-line signature verification. In: *23rd International Conference Image and Vision Computing New Zealand (IVCNZ)*; 2008. p. 1–6. https://doi.org/10.1109/IVCNZ.2008.4762072

[5] Wajid R, Malik H, Malik MI. Offline signature verification using local binary pattern features. *International Journal of Pattern Recognition and Artificial Intelligence*. 2014;28(7):1456003. https://doi.org/10.1142/S0218001414560036

[6] Ooi SY, Teoh ABJ, Pang YH. Radon transform based approach for offline signature verification. *Expert Systems with Applications*. 2013;40(17):6838–6846. https://doi.org/10.1016/j.eswa.2013.06.042

[7] Hafemann LG, Sabourin R, Oliveira LS. Learning features for offline handwritten signature verification using deep convolutional neural networks. *Pattern Recognition*. 2017;70:163–176. https://doi.org/10.1016/j.patcog.2017.05.012

[8] Dey S, Dutta A, Toledo JI, Ghosh SK, Lladós J, Pal U. SigNet: Convolutional Siamese network for writer independent offline signature verification. *Pattern Recognition Letters*. 2019;128:182–189. https://doi.org/10.1016/j.patrec.2017.07.024

[9] Guerbai A, Chibani Y, Hadjadji B. The effective use of one-class SVM classifier for handwritten signature verification based on writer-independent parameters. *Pattern Recognition*. 2015;48(1):103–113. https://doi.org/10.1016/j.patcog.2014.07.015

[10] Lundberg SM, Lee S-I. A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*. 2017;30:4765–4774.

[11] Kiran P, Parameshachari BD, Yashwanth J, Bharath KN. Offline signature recognition using image processing techniques and back propagation neuron network system. *SN Computer Science*. 2021;2(3):196. https://doi.org/10.1007/s42979-021-00591-y

[12] Poddar J, Parikh V, Bharti SK. Offline signature recognition and forgery detection using deep learning. *Procedia Computer Science*. 2020;170:610–617. https://doi.org/10.1016/j.procs.2020.03.133

[13] Hashim Z, Ahmed HM, Alkhayyat AH. A comparative study among handwritten signature verification methods using machine learning techniques. *Scientific Programming*. 2022;2022:8170424. https://doi.org/10.1155/2022/8170424

[14] Wang Z, Muhammat M, Yadikar N, Aysa A, Ubul K. Advances in offline handwritten signature recognition research: A review. *IEEE Access*. 2023;11:120222–120236. https://doi.org/10.1109/ACCESS.2023.3326471

IRE JOURNALS

ICONIC RESEARCH AND ENGINEERING JOURNALS

AN INTERNATIONAL OPEN ACCESS · PEER-REVIEWED · REFEREED JOURNAL

Certificate of Publication

This is to certify that

MADIHA KAUNAIN

has published the research paper entitled

FORGEXPLAIN: AN EXPLAINABLE MACHINE LEARNING SYSTEM FOR OFFLINE SIGNATURE FORGERY DETECTION

PUBLISHED IN	VOLUME & ISSUE	MONTH OF ISSUE
e-Journal	Volume 10 Issue 4	October 2026

PAPER ID

1723878

www.irejournals.com

ISSN

2456-8880

Sanket Patel

EDITOR-IN-CHIEF

IRE Journals

IRE JOURNALS

ICONIC RESEARCH AND ENGINEERING JOURNALS

AN INTERNATIONAL OPEN ACCESS · PEER-REVIEWED · REFEREED JOURNAL

Certificate of Publication

This is to certify that

NALINI M

has published the research paper entitled

FORGEXPLAIN: AN EXPLAINABLE MACHINE LEARNING SYSTEM FOR OFFLINE SIGNATURE FORGERY DETECTION

PUBLISHED IN	VOLUME & ISSUE	MONTH OF ISSUE
e-Journal	Volume 10 Issue 4	October 2026

PAPER ID

1723878

www.irejournals.com

ISSN

2456-8880

Sanket Patel

EDITOR-IN-CHIEF

IRE Journals

IRE JOURNALS

ICONIC RESEARCH AND ENGINEERING JOURNALS

AN INTERNATIONAL OPEN ACCESS · PEER-REVIEWED · REFEREED JOURNAL

Certificate of Publication

This is to certify that

NOOR AYESHA

has published the research paper entitled

FORGEXPLAIN: AN EXPLAINABLE MACHINE LEARNING SYSTEM FOR OFFLINE SIGNATURE FORGERY DETECTION

PUBLISHED IN	VOLUME & ISSUE	MONTH OF ISSUE
e-Journal	Volume 10 Issue 4	October 2026

PAPER ID

1723878

www.irejournals.com

ISSN

2456-8880

Sanket Patel

EDITOR-IN-CHIEF

IRE Journals

IRE JOURNALS

ICONIC RESEARCH AND ENGINEERING JOURNALS

AN INTERNATIONAL OPEN ACCESS · PEER-REVIEWED · REFEREED JOURNAL

Certificate of Publication

This is to certify that

R. SAARANYA M. E.

has published the research paper entitled

FORGEXPLAIN: AN EXPLAINABLE MACHINE LEARNING SYSTEM FOR OFFLINE SIGNATURE FORGERY DETECTION

PUBLISHED IN	VOLUME & ISSUE	MONTH OF ISSUE
e-Journal	Volume 10 Issue 4	October 2026

PAPER ID

1723878

www.irejournals.com

ISSN

2456-8880

Sanket Patel

EDITOR-IN-CHIEF

IRE Journals